

IT PROJECT 700

Hospital Management System (HMS)

30 June 2026

Document Control

Qualification	BSc. in IT
Module	IT Project 700
Title	IT Project: Hospital Management System (HMS)
DocRef	2025M2ITP700M
Creation Date	16 March 2026

Group/Contributors List

Name	Student Number	Signed	Comments
Kudzai Kazenga	402204080		
Joseph Kgafela	402111436		
Nonkululeko Maluleke	402309629		
Zanele Mabunda	402101358		
Sinegugu Ngobese	402200826		

Version History

Version	Date	Status	Amended by	Summary of Changes
v1.0.0	16 March 2026	Draft	Kudzai Kazenga	Document Created Phase 1
V1.1.2	23 March 2026	Draft	Group (MS Teams)	Section 1.3 Key Deliverables/Value proposition
V1.1.3	13 March 2026	Draft	Group (MS Teams)	Section 2: Planning Phase
V1.1.4	11 May 2026	Draft	Group (MS Teams)	Section 3: Analysis Phase
V1.1.5	8 June 2026	Draft	Group (MS Teams)	Section 4: System Design Phase
V1.1.6	29 June 2026	Final	Group (MS Teams)	Section 5: Implementation

Supervisor

Name	Email
Royal Goronga	RoyalG@richfield.ac.za

Table of Contents

1	Introduction/Objectives.....	7
1.1	Purpose	7
1.2	Objectives.....	7
1.3	Key Deliverables/Value Proposition	7
1.4	Programming Language	8
1.5	SDLC Implementation Model.....	8
2	Planning Phase	9
2.1	Identification of Need.....	9
2.2	Preliminary Investigation	9
2.3	Feasibility Study	10
2.4	Project Planning Report.....	12
2.4.1	Project Scope	12
2.4.2	Work Breakdown Structure (WBS).....	12
2.4.3	Task Dependencies	13
2.4.4	Time Estimation	13
2.4.5	Roles and Responsibilities	13
2.4.6	Risk Management	13
2.4.7	Communication Plan	13
2.4.8	Constraints and Assumptions	14
2.4.9	Tools and Technologies.....	14
2.4.10	Success Criteria	14
2.5	Project Scheduling.....	14
2.6	Software Requirements Specification	16
2.6.1	Functional Requirements	16
2.6.2	Non-Functional Requirements.....	16
2.6.3	Solution Architecture Design	17
2.6.4	Technology Platform	17
2.7	Data Models 2.7.1 High-Level Business Processes	18
2.7.2	High-Level Data Model.....	18
2.7.3	Data Schema (high-level)	19
2.7.4	Data Elements – High-Level Data Dictionary.....	19
2.7.5	Software Engineering Paradigm Applied	20
3	Analysis Phase	21
3.1	Information Gathering Methodology	21
3.1.1	Observation Method.....	21
3.1.2	Participatory Method.....	22
3.1.3	Interview Method	22

3.2	Analysis of Existing Systems	22
3.3	Data Analysis (Data Integrity and Constraints)	23
3.3.1	Types of Data in the Proposed System	24
3.3.2	Data Integrity	24
3.3.3	Constraints in the Proposed System	25
3.3.4	Conclusion	25
3.4	Weakness of the Current System	25
3.5	Analysis of the Proposed System (Functional Requirements)	26
3.5.1	FR-01: Patient Data Management	26
3.5.2	FR-02: Medicine/Prescriptions Issuance	28
3.5.3	FR-03: Appointment Scheduling	30
3.5.4	FR-04: Billing Management	31
3.5.5	FR-05: Risk Management Tracking	34
3.5.6	FR-06: Doctor/Nurse Management	37
3.5	Non-Functional Requirements	39
3.6.1	Reliability & Sync Logic (NFR-3)	39
3.6	Data Modelling for Proposed System	40
3.7.1	Considerations	43
3.7.2	Integrity enforcement	43
3.7.3	Assumptions	43
4	System Phase	44
4.1	Introduction	44
4.2	System Design	44
4.3	Architectural Design	45
4.3.1	Software Architectural Design	45
4.3.2	Hardware Architectural Design	46
4.3.3	Network Architectural Design	46
4.3.4	Class Diagram	47
4.4	Physical Design	48
4.5	Database Design	49
4.5.1	Physical Design	49
4.5.2	Third Normal Form (3NF) Compliance	49
4.5.3	Logical Design	49
4.6	Program Design	54
	Module Summary	55
4.7	Interface Design	56
4.8	Security and Back-up Design	63

4.8.1 Software Security	63
4.8.2 Backup and Recovery Design.....	66
5 Implementation Phase	69
5.1 Introduction	69
5.2 Coding	69
5.3 Testing	70
5.4 System Testing (Test Case, Evaluation of the testing results)	72
5.5 Installation	78
5.5.1 System Requirements	78
5.5.2 Installation Steps.....	78
6 References	80
7 Glossary	81
8 Appendices B: Program Source Files.....	86

List of Figures

Figure 0-1: FR-01 Sequence Diagram – Admin	27
Figure 0-2: FR-01 Sequence Diagram – Patient Update Record.....	28
Figure 0-3: FR-02 Sequence Diagram.....	29
Figure 0-4: FR-03 Sequence Diagram.....	30
Figure 0-5: In-Patient Billing Report.....	31
Figure 0-6: Generate Invoice.....	32
Figure 0-7: Payment Methods	32
Figure 0-8: Process Payment	33
Figure 0-9: FR-05 Sequence Diagram – Inventory Management	34
Figure 0-10: FR-05 Staff Shift Management.....	35
Figure 0-11: FR-06 Sequence Diagram.....	38
Figure 6-1: HMS ERD.....	40
Figure 13: Software Architecture Design	45
Figure 14: HMS Physical Design	48
Figure 15: Landing Page UI	56
Figure 16: Login UI.....	57
Figure 17: Patient Registration UI.....	57
Figure 18: Schedule Appointment UI.....	58
Figure 19: Patient Dashboard	58
Figure 20: Doctor’s Dashboard	59
Figure 21: Nurse Dashboard.....	59
Figure 22: Finance/Billing Dashboard.....	60
Figure 23: Inventory Management Dashboard	60
Figure 24: Suppliers Dashboard	61
Figure 25: Inventory PO	61
Figure 26: Inventory Stock	62
Figure 27: Audit Logs Dashboard	62
Figure 28: Admin Dashboard	63
Figure 29: HMS Log in Page.....	79

List of Tables

Table 1: Languages	8
Table 2-2: Work Breakdown Structure.....	12
Table 3: Patient Profile	28
Table 4: Prescriptions Data Store	29
Table 5: Appointments	31
Table 6: Billing Data	33
Table 7: Inventory Data	36
Table 8: Supplier Data	36
Table 9: Purchase Oder Data	36
Table 10: Stock Receipt Data	36
Table 11: FR-06 Data	38
Table 12: NFR Analysis Map	39
Table 13: Offline Sync Data	39
Table 14: HMS Database Relationships	43
Table 15: Software Architecture Layers.....	45
Table 16:Hardware Design Areas.....	46
Table 17:Network Architecture	47
Table 18: HMS Class Diagram.....	48
Table 19: Patient Entity	50
Table 20: Doctor Entity	50
Table 21: Nurse Entity.....	50
Table 22: Staff Entity.....	50
Table 23: Appointment Entity	51
Table 24:Prescription Entity	51
Table 25: Billing records Entity	51
Table 26: Inventory entity	52
Table 27: Supplier entity	52
Table 28: Purchase Order Entity.....	52
Table 29: Stock receipt entity	52
Table 30: Staff shift entity	53
Table 31: Audit Log entity	53
Table 32: SyncLog Entity	53
Table 33: Logical relationship map	54
Table 34: RBAC matrix.....	64
Table 35: Audit Log Compliance matrix.....	65
Table 36: Regulatory Compliance	66
Table 37: Back-up Strategy	66
Table 38: Back-up schedule.....	68
Table 39: Development Envriionment.....	69
Table 40: HMS Test Cases	77
Table 41: System Requirements	78
Table 42: Sample Profiles Test data	79

1 Introduction/Objectives

1.1 Purpose

The primary objective of the project is to design and implement a web-based platform, the **Hospital Management System (HMS)**, for the digitalization of patient records and streamline hospital workflows efficiently. The HMS is designed to replace fragmented records through the centralization of hospital data and processes, migrating to a paperless solution for the day-to-day running of the hospital.

1.2 Objectives

The core objective of the project is to achieve the following:

- **Data Management**
Digitizing the patient/doctor records, reducing human errors and maintaining a comprehensive audit trail for accountability
- **Medicines Management**
Digitizing the issuance of prescriptions and medicines in line with regulations
- **Efficient Scheduling**
Streamline the patient/doctor appointments
- **Data Privacy and Security**
Maintain high levels of data privacy and security to safeguard sensitive data records
- **Regulatory Compliance**
Ensure compliance to regulatory laws under POPIA (Protection of Personal Information and Act), NHA (National Health Act), and HPCSA (Health Professionals Council of SA)
- **Billing Transparency**
Facilitate patient billing models seamlessly

1.3 Key Deliverables/Value Proposition

To tackle the key issues in the current existing systems offered in both private and public landscapes, our HMS will delivery:

1. **Offline-First Processing** - ensuring the system still works with interrupted/limited internet and to cater for downtime/offline events and allow continuity of operations and increase service levels ratings
2. **Interoperability** - enables sharing of patient data on a needs basis with any system in the health care sector, both within the ecosystem or externally.
3. **Risk Management System Analysis** – enabling early warning tracking systems on inventory, resources, and personnel to avert any risks identified by the system
4. **Patient-centric** – offering direct access to patients to review and share their data including scheduling, historic data and doctors notes.

1.4 Programming Language

The selection below supports a web-based system that is cross-platform compatible, lightweight and mobile friendly

Component	Definition
Front-end	HTML, CSS, JavaScript
Backend	PHP
Database	MySQL (Relational Database)

Table 1: Languages

1.5 SDLC Implementation Model

For this project, the **Agile Iterative Model** has been chosen, implemented through **SCRUM methodology**.

This approach is ideal for a **Hospital Management System** because it accommodates continuous development and refinement of complex modules while delivering value in each iteration based on feedback.

Furthermore, this approach aligns with the Agile Manifesto by utilizing regular feedback loops, enables adaptability in event of changing requirements and adherence to regulatory regulations withing the South African healthcare services.

2 Planning Phase

2.1 Identification of Need

Healthcare institutions generate large volumes of information related to patients, medical staff, appointments, treatments, billing, and inventory. Many hospitals still rely on manual record-keeping systems or partially digitized processes, which often result in inefficiencies, delays, and data management challenges. These traditional systems make it difficult to store, retrieve, and manage patient information accurately and efficiently.

The need for a Hospital Management System (**HMS**) arises from the necessity to streamline hospital operations and improve the overall quality of healthcare services. A computerized system can centralize hospital information, allowing authorized staff to quickly access patient records, manage appointments, track treatments, and handle billing processes efficiently.

An HMS can significantly reduce paperwork, minimize human errors, and improve communication between different hospital departments such as reception, pharmacy, laboratory, and administration. Additionally, the system enhances data security by implementing role-based access controls that restrict sensitive medical information to authorized personnel only.

Another important need for such a system is efficient resource management. Hospitals must manage various resources including medical staff, hospital beds, medicines, and equipment. A well-designed system enables administrators to monitor these resources in real time and make informed decisions. Furthermore, the growing demand for quality healthcare services requires hospitals to maintain accurate and up-to-date patient records. A digital management system ensures better patient care by providing doctors with quick access to patient histories, test results, and treatment records.

Therefore, the development of a Hospital Management System is necessary to improve operational efficiency, enhance patient care, reduce administrative workload, and support better decision-making within healthcare institutions.

2.2 Preliminary Investigation

A preliminary investigation is conducted to determine whether developing a Hospital Management System is feasible and beneficial for the healthcare facility. This stage focuses on understanding the current system, identifying existing problems, and evaluating potential solutions.

Understanding the Current System

Most hospitals operate with a combination of manual paperwork and basic digital tools. Patient information is often recorded in files, spreadsheets, or isolated databases. This fragmented system leads to difficulties in accessing information quickly and increases the risk of data loss or duplication.

Staff members such as receptionists, doctors, and administrators may spend a significant amount of time searching for records, updating files, or transferring information between departments. These inefficiencies negatively impact the overall performance of the hospital and can delay patient care.

Problem Identification

The investigation of the current system reveals several key problems:

- Time-consuming processes: Manual record management slows down hospital operations.
- Data redundancy and errors: Repeated data entry increases the risk of mistakes.
- Poor data accessibility: Retrieving patient records can be difficult and time-consuming.
- Limited coordination between departments: Information sharing between departments is inefficient.
- Security risks: Physical records can be lost, damaged, or accessed by unauthorized individuals.

Proposed System Overview

Based on the investigation, the proposed Hospital Management System will provide the following features:

- Patient registration and record management
- Doctor and staff management
- Appointment scheduling
- Billing and payment processing
- Secure login with role-based access control

This system will enable hospitals to operate more efficiently, improve service delivery, and ensure accurate management of healthcare data.

2.3 Feasibility Study

Technical Feasibility

The proposed Hospital Management System (HMS) is technically feasible because it uses reliable, widely adopted, and cost-effective technologies.

The system will be developed using:

- Front-end: HTML, CSS, JavaScript (for user-friendly interfaces)
- Back-end: PHP (for handling business logic)
- Database: MySQL (for structured data storage)

These technologies are:

- Easy to deploy and maintain
- Compatible across different devices (desktop, tablet, mobile)
- Supported by a large developer community

The system will also support:

- Offline-first functionality, allowing operations to continue during internet downtime
- Interoperability, enabling integration with other healthcare systems
- Security mechanisms, such as encryption and user authentication, to protect sensitive data

Overall, the required hardware and software are affordable and readily available, making the system technically viable.

Operational Feasibility

The HMS is designed to improve hospital and clinic operations by replacing manual, paper-based processes with a digital system.

How the system improves operations:-

- **Data Management:** Patient and doctor records will be digitized, reducing errors and improving accuracy. Audit trails will help track all changes for accountability.
- **Medicines Management:** Prescriptions and medicine dispensing will be managed electronically, ensuring compliance with healthcare regulations.
- **Efficient Scheduling:** Appointment booking will be streamlined, reducing long queues and improving patient experience.
- **Patient-Centric Approach:** Patients can access their records, appointments, and medical history, giving them more control over their healthcare.
- **Risk Management:** The system will monitor inventory, staff, and resources to identify potential issues early.

Staff Adaptation:-

- The system will include a simple and intuitive interface
- Training sessions can be provided for hospital staff
- Minimal disruption is expected during implementation

This ensures that the system will be accepted and effectively used by healthcare professionals.

Economic Feasibility

The HMS is economically viable, offering long-term financial benefits despite initial setup costs.

Costs involved:

- System development (programming and design)
- Hardware (computers, servers if needed)
- Training for staff
- Maintenance and updates

Benefits:

- Reduced paperwork and administrative costs
- Fewer medical errors (which can be costly)
- Improved efficiency and faster service delivery
- Better resource management (medicines, staff, equipment)
- Increased patient satisfaction, leading to a better reputation

Because the system uses open-source technologies (PHP and MySQL), licensing costs are minimal, making it a cost-effective solution.

Legal and Regulatory Feasibility

The system will comply with South African healthcare regulations, including:

- POPIA (Protection of Personal Information Act): Ensures patient data is securely stored and processed.
- National Health Act (NHA): Governs healthcare standards and patient rights.
- HPCSA (Health Professions Council of South Africa): Ensures ethical handling of patient information by healthcare professionals.

Security features such as:

- Role-based access control
- Data encryption
- Secure login authentication

2.4 Project Planning Report

2.4.1 Project Scope

The Hospital Management System (HMS) aims to digitize hospital processes such as patient management, appointments, billing, and reporting. Phase 1 focuses on core functionalities required for daily operations, while Phase 2 introduces advanced features such as automation, analytics, and third-party integrations.

2.4.2 Work Breakdown Structure (WBS)

Level 1	Level 2	Level 3
Planning	Requirement Gathering	-
Planning	Feasibility Study	-
Design	Database Design	ERD Schema
Design	UI Design	-
Development	Backend	-
Development	Frontend	-
Testing	Unit Testing	-
Testing	Integration Testing	-
Deployment	Installation	-

Table 2-2: Work Breakdown Structure

2.4.3 Task Dependencies

Project tasks are interdependent. Design must be completed before development begins, while testing occurs after development. Proper sequencing ensures efficiency and reduces rework.

2.4.4 Time Estimation

Task	Duration
Planning	3 days
Design	5 days
Backend Design	8 days
Frontend Design	6 days
Testing	7 days
Deployment	2 days

Table 2-2: Time Estimation

2.4.5 Roles and Responsibilities

Role	Responsibility
Developer	System development and coding
Tester	Bug detection and validation
Administrator	System management
User	Feedback and usability testing

Table 2-3: Roles and Responsibilities

2.4.6 Risk Management

Risk	Impact	Mitigation
System Failure	High	Regular Backup
Poor UI	Medium	User Testing
Security Breach	High	Encryption

Table 2-4: Risk Management

2.4.7 Communication Plan

Stakeholder	Method	Frequency
Development Team	Meetings	Daily
Administrator	Reports	Weekly
Users	Feedback	Testing Phase

Table 2-5: Communication Plan

2.4.8 Constraints and Assumptions

Constraints include limited time, budget, and resources. Assumptions include basic user computer literacy, stable internet access, and availability of system infrastructure.

2.4.9 Tools and Technologies

- Frontend: HTML, CSS, JavaScript.
Backend: Python/PHP/Java.
- Database: MySQL.
- Tools: VS Code, MySQL Workbench.
Version control using Git is recommended for collaboration.

2.4.10 Success Criteria

The system will be successful if it operates without errors, supports all core functionalities, maintains fast response times, ensures data security, and achieves user satisfaction.

2.5 Project Scheduling

This table outlines project phases, tasks, predecessors, and durations.

It provides a structured breakdown of analysis, design, implementation, UAT, and deployment.

Phase	ID	Task Name	Predecessor	Duration
Analysis	A1	Stakeholder Interviews and Workflow Mapping	-	5 Days
	A2	Compliance Audit (POPIA, NHA, HPCSA)	A1	3 Days
	A3	Requirement Specification Document	A2	2 Days
System Design	D1	Database Schema Design (MySQL)	A1, A2	4 Days
	D2	UI/UX Wireframing and Prototyping	D1	3 Days
	D3	Offline-First and Sync Architecture	D2	4 Days
Implementation	I1	Sprint 1: Core Patient and Doctor Modules	D2, D3	10 Days
	I2	Sprint 2: Billing and Meds Management	I1	10 Days
UAT	U1	Security, Interoperability and Quality Assurance	I1, I2	5 Days

	U2	Refinement and SME Sign Off		2 Days
Deployment		Training documentation and Handover		2 Days

Table 4-3

The PERT chart visualizes task dependencies and timelines.

It highlights the critical path and sequencing from analysis through deployment.

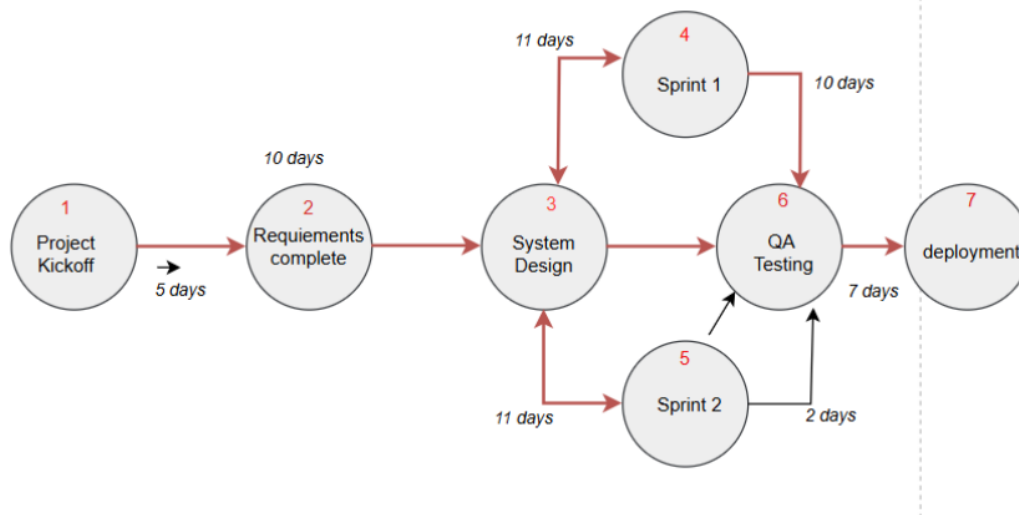


Figure 2.1: PERT Chart

The Gantt chart presents tasks against a timeline with durations. It shows overlaps, dependencies, and progress flow across project phases.

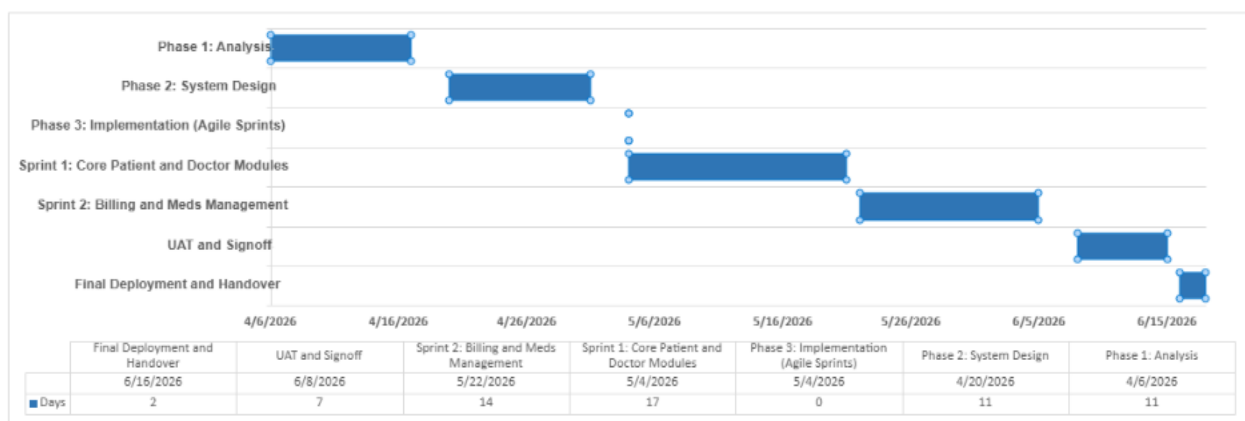


Figure 2.2: Gantt Chart

2.6 Software Requirements Specification

2.6.1 Functional Requirements

These requirements below define the specific behaviours and services the HMS must provide.

Req. ID	Requirement	Description	Stakeholder Needs
FR-01	Patient Data Management	Patient profile and records management	Replaces paper-based files.
FR-06	Doctor/Nurse Management	The system must manage internal 16 profiles and activities	Enhanced internal processes
FR-02	Medicine/Prescriptions Issuance	Digital tracking system of prescriptions and dispensing as regulated	Must track dosage and history.
FR-03	Appointment Scheduling	Create and manage patient/doctor appointments	Includes direct patient access.
FR-04	Billing Management	Generate and manage patient billing and statements	Must support various billing models.
FR-05	Risk Management Tracking	Inventory: An early warning tracking system to manage stock and reordering HR: manage doctor/nurse shift and staffing resources	Used to avert operational risks.
TR-01	Data Migration	Transition Requirement(TR): Migrate existing fragmented physical and digital records into the new centralized HMS database.	Essential for "Go-Live" phase.

Table 4-2: Functional Requirements

2.6.2 Non-Functional Requirements

These define the quality attributes and constraints of the system.

ID	Scope	Requirement Description
NFR-1	Security and Compliance	The system must comply fully with health care and data handling regulations such as POPIA, NHA, HPCSA maintain high levels of data encryption such as 2FA, and strict access controls.

ID	Scope	Requirement Description
NFR-2	Performance	The core system functionality, reporting and data entry must process in faster response times within reasonable times
NFR-3	Reliability	The system must maintain core functionality and data entry during internet and/or limited downtime to ensure continuity of operations within an acceptable window period during prescribed operating hours
NFR-4	Usability, Interoperability	The system UI design must be consistent to branding standards and maintain a user-friendly interface. The system must be compatible to existing health systems for data sharing and external integration
NFR-5	Scalability	The system infrastructure and technological architecture must support growth vertically and horizontally, including 3rdParty integrations for added functionalities.
NFR-6	Auditability	Logging of user activity and secured access logs for transparency and extraction of full audit trail.

Table 2-8: Non-Functional requirements

2.6.3 Solution Architecture Design

The HMS is designed to focus on a **Patient-Centric** model with the following entities in scope

- Patients - data access, appointment scheduling, medicines
- Doctors - clinical notes, prescriptions
- Nurses - patient care and attending
- Admin/Billing – records management, finance
- Regulatory Bodies - compliance and auditing

2.6.4 Technology Platform

The system is built as a lightweight, cross-platform web application.

- **Front-end:** HTML, CSS, and JavaScript.
- Backend: PHP.
- **Database:** MySQL (Relational Database).
- **Network Architecture:** Hybrid Cloud-Edge model to support **Offline-First Processing**.
 - **Connectivity:** HTTPS/TLS encryption for secure data transit between the local hospital local area network (LAN) and the Azure-hosted cloud environment.
 - **Synchronization:** Local caching mechanisms (Service Workers/Indexed DB) to allow continuity of operations during limited internet, downtime or temporary power failures by switching to mobile/hand-held devices

- **Interoperability:** Enable a secure API gateway to enable sharing of patient data with external healthcare ecosystems on a needs basis and plug in external parties for added functionality and capability

2.7 Data Models

2.7.1 High-Level Business Processes

1. **Patient Intake:** Digital registration and verification under POPIA standards.
2. **In-Patient Consultation:** Doctor records notes and issues digital prescriptions.
3. **Resource Management:** System monitors inventory levels to trigger early risk warnings.
4. **Billing:** Seamless transition from clinical care to financial settlement.

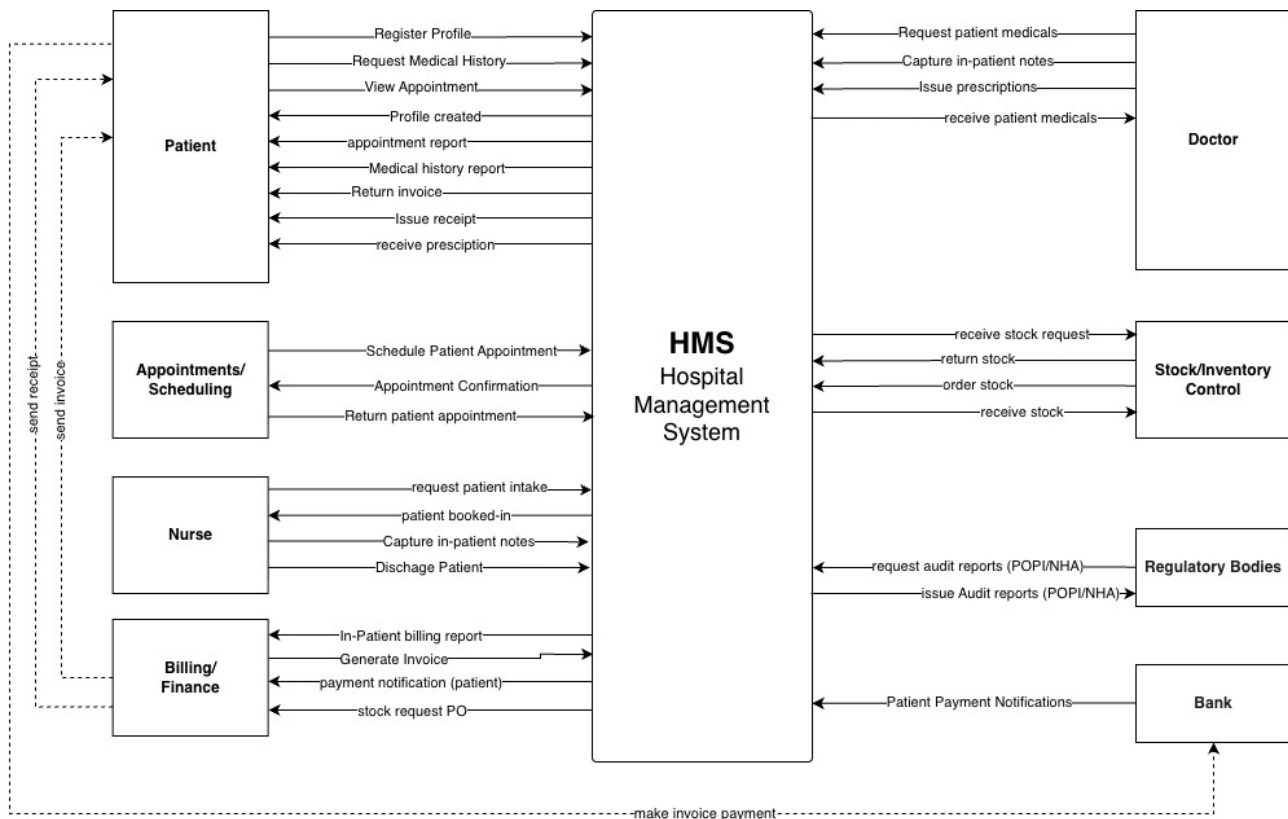


Figure 2.3: Context Diagram

2.7.2 High-Level Data Model

The data model is **Relational**, managed via **MySQL**.

1. **Core Entities:** Patient, Doctor, Nurse, Appointment, Prescription, Inventory, Staff Shift, Billing Record, and Audit Log.
2. **Relationships:** The following relationships exist in the relational DB for HMS

One-to-Many

- Patient to Appointment: A patient can have many appointments over time.
- Doctor to Appointment: A doctor attends to multiple scheduled patients.
- Patient to Prescription: A patient can be issued multiple prescriptions over time

- Patient to Billing Record: A patient can have multiple billing invoices.

Many-to-One

- Staff (Nurse/Doctor) to Staff Shift: Multiple staff members are assigned to specific shift time allocation.

One-to-One

- System Action to Audit Log: Each unique system interaction triggers a corresponding audit entry in the Audit Log file

2.7.3 Data Schema (high-level)

Entity	Description
Patient	Stores patient personal and medical information
Doctor	Stores doctor details and specialization
Nurse	Stores nurse details
Appointment	Links patient and doctor for scheduling
Prescription	Medication issued to patient
Inventory	Tracks medicine stock levels
Staff Shift	Tracks staffing resources
Billing Record	Handles patient billing and payments
Audit Log	Tracks all system activities

Table 2-9: Data Schema

2.7.4 Data Elements – High-Level Data Dictionary

Patient - PatientId (PK), PatientName, PatientIdNumber, PatientContact, PatientMedicalNotes

Doctor - DoctorId (PK), DocName, DocSurname, DocSpecialization, DocLicenseNoHPCSA

Nurse - NurseId (PK), Name, Surname, Department, ShiftId (FK), NurseLicenseNo

Appointment - ApptId (PK), PatientId (FK), DoctorId (FK), Date, Status

Prescription - RxId (PK), PatientId (FK), DoctorId (FK), Medication, Dosage, Quantity

Inventory - ItemId (PK), ItemName, StockLevel, ReorderPoint, UnitPrice, AlertId, AlertDate

Staff – StaffId (PK), DoctorId (FK), NurseId (FK), Role, Status

Staff_Shift - ShiftId (PK), StaffId (FK), StartTime, EndTime

Billing_Record - BillId (PK), PatientId (FK), Amount, PaymentStatus, Date

Supplier – SupplierID (PK), SupplierName, ContactEmail, Phone

PurchaseOrder – PurchaseOrderId (PK), ItemId (FK), SupplierId (FK), Quantiy, TotalCost, Status, OrderDate

StockReceipt – ReceiptId (PK), PurchaseOrderId (FK), QuantityReceived, ReceivedDate

SyncLog – LogId (PK), SyncStatus, Timestamp, IP_Address

Audit_Log - LogId (PK), StaffId, Action, Timestamp, IP_Address,

2.7.5 Software Engineering Paradigm Applied

As stated in the initial proposal, the most effective paradigm is the Agile Methodology using the Scrum framework.

Advantages

- Iterative Delivery
- Flexibility
- Continuous Integration

Proposed Sprints:

- Sprint 1: Patient and Nurse onboarding (User Authentication)
- Sprint 2: Doctor and Admin Onboarding (User Authentication)
- Sprint 3: Inventory Management: Medicines
- Sprint 4: Doctor + Nurse in-patient notes and prescriptions
- Sprint 5: Admin Billing (invoices) Patient scheduling.
- Sprint 6: Patient scheduling/appointments
- Sprint 7: Backend: Offline-first logic

3 Analysis Phase

The Hospital Management System (HMS) project was initiated to address the challenges faced by healthcare institutions in managing patient information, appointments, billing, prescriptions, and hospital resources efficiently. Many hospitals still rely on manual or partially computerized systems, which often result in delays, data duplication, poor communication between departments, and security risks.

The proposed HMS aims to provide a centralized web-based platform that digitizes hospital operations and improves service delivery. The system focuses on enhancing operational efficiency, reducing paperwork, improving patient care, and ensuring compliance with South African healthcare regulations such as POPIA, the National Health Act (NHA), and HPCSA requirements. The project follows the Agile Software Development Life Cycle (SDLC) model using the Scrum methodology.

This approach allows continuous feedback, flexibility, and iterative improvement throughout the development process. The technologies selected for the system include HTML, CSS, JavaScript, PHP, and MySQL, which support a lightweight, secure, and cross-platform solution.

3.1 Information Gathering Methodology

Information gathering is an important phase in system development because it helps developers understand the current system, identify problems, and determine user requirements. For the Hospital Management System project, several data collection methods were used, including observation, participatory investigation, and interviews.

3.1.1 Observation Method

Observation involved studying how hospital staff currently perform daily activities such as patient registration, appointment scheduling, record keeping, billing, and medicine management. Through direct observation, it was identified that many processes are manual and paper-based, which causes delays, inefficiencies, and difficulty in retrieving patient information. Staff members were observed spending significant time searching for files and transferring information between departments.

Observation also helped identify operational challenges such as:

- Long patient waiting times
- Repeated data entry
- Misplaced or duplicated records
- Poor coordination between departments
- Limited access to patient information

This method provided a clear understanding of the weaknesses in the existing system and highlighted areas requiring automation and improvement.

3.1.2 Participatory Method

The participatory method involved engaging hospital stakeholders such as administrators, receptionists, doctors, nurses, and patients during the investigation process. Stakeholders contributed their opinions, challenges, and expectations regarding the proposed system.

Through participation, users explained the importance of features such as:

- Patient record management
- Appointment scheduling
- Billing management
- Digital prescriptions
- Inventory monitoring
- Secure access control

The participatory approach ensured that the proposed HMS was designed according to the actual needs of users and healthcare professionals. It also encouraged collaboration and improved user acceptance of the new system.

3.1.3 Interview Method

Interviews were conducted with hospital staff and management to gather detailed information about the current processes and challenges experienced within the healthcare environment. Structured and semi-structured interview questions were used to obtain information about workflow procedures, data management, security concerns, and operational requirements.

The interviews revealed several major issues in the current system, including:

- Time-consuming manual processes
- Data redundancy and human errors
- Poor accessibility of records
- Weak communication between departments
- Risks associated with physical file storage

Interviews also helped identify important system requirements such as data security, regulatory compliance, interoperability, offline functionality, and patient-centered services.

Overall, the use of observation, participatory investigation, and interviews provided comprehensive information required for the successful analysis and design of the Hospital Management System.

3.2 Analysis of Existing Systems

The analysis of the existing hospital system revealed that most operations are performed manually or through isolated digital tools such as spreadsheets and standalone databases. Patient records are stored in physical files, making it difficult to retrieve information quickly and accurately.

The current system has several weaknesses that negatively affect hospital performance and patient care. These include:

Manual Record Management

Patient information is recorded manually, which increases paperwork and administrative workload. Manual filing systems are slow and prone to errors, duplication, and data loss.

Poor Data Accessibility

Hospital staff often experience delays when retrieving patient records because files are stored in different departments or locations. This affects decision-making and patient treatment processes.

Lack of Departmental Integration

Departments such as reception, pharmacy, laboratory, billing, and administration operate independently with limited information sharing. This causes communication gaps and inefficiencies within the hospital.

Security Risks

Physical records can easily be damaged, lost, or accessed by unauthorized individuals. The current system lacks strong security mechanisms such as role-based access control and encryption.

Inefficient Appointment and Billing Processes

Appointment scheduling and billing procedures are handled manually, leading to long waiting times, missed appointments, billing inaccuracies, and poor patient experience.

Limited Resource Management

The hospital faces challenges in managing medicines, equipment, staff shifts, and inventory efficiently because there is no centralized monitoring system.

Conclusion of System Analysis

The analysis shows that the existing system is inefficient, time-consuming, and unable to meet the growing demands of modern healthcare services. Therefore, the implementation of the proposed Hospital Management System is necessary to improve operational efficiency, enhance data security, streamline communication, reduce human errors, and provide better patient care. The proposed HMS will centralize hospital operations into a secure digital platform that supports effective healthcare management.

3.3 Data Analysis (Data Integrity and Constraints)

Data analysis is an important part of the Hospital Management System (HMS) because it handles large volumes of sensitive and critical information daily. Hospitals depend on accurate patient records, appointment schedules, billing information, prescriptions, laboratory results, and staff details to provide effective healthcare services. Poorly managed data can result in delays, incorrect treatment, billing errors, and security breaches.

The proposed Hospital Management System will use a centralized database to ensure that all hospital departments can access accurate and up-to-date information in real time. The purpose of this section is to analyse the data requirements of the proposed system and explain the mechanisms that will be used to maintain data integrity and enforce constraints.

3.3.1 Types of Data in the Proposed System

- **Patient Data**
This includes all information related to patients receiving treatment at the hospital. Examples include Patient ID, full name, gender, date of birth, contact details, medical history, allergies, blood group, and admission records.
- **Doctor Data**
The system will store information related to doctors and specialists, such as Doctor ID, specialization, department, schedules, and contact details.
- **Appointment Data**
The system will manage appointment scheduling between patients and doctors. Data includes appointment ID, appointment date, patient ID, Doctor ID, and appointment status.
- **Billing and Payment Data**
The system will manage invoices, consultation fees, laboratory charges, medication costs, payment status, and payment methods.
- **Prescription and Pharmacy Data**
Prescription details such as medication name, dosage, quantity prescribed, and prescribing doctor will be stored in the system.
- **Staff and Administration Data**
The system will also store employee details, including staff ID, role, department, attendance records, and login credentials.

3.3.2 Data Integrity

- **Entity Integrity**
Each table in the database will contain a unique primary key, such as Patient ID, Doctor ID, or Appointment ID, to prevent duplicate records.
- **Referential Integrity**
Relationships between tables will be maintained using foreign keys. For example, appointments must reference valid patient and doctor records.
- **Domain Integrity**
Only valid values will be accepted in the system. For example, age must be greater than zero, and phone numbers must contain valid digits.
- **Data Validation**
- Validation measures such as mandatory fields, range validation, and format validation will reduce incorrect data entry.
- **Data Consistency**
- Standardized data formats will be used throughout the system to improve reporting and reduce confusion.
- **Security and Confidentiality**
- Security mechanisms, including password encryption, user authentication, role-based access control, and audit logs, will protect sensitive patient information.
- **Backup and Recovery**

- Daily backups and disaster recovery procedures will ensure that hospital data can be recovered in case of system failure or accidental deletion.

3.3.3 Constraints in the Proposed System

- **Primary Key Constraints**
Every table will contain a unique identifier to prevent duplicate entries.
- **Foreign Key Constraints**
Foreign keys will maintain relationships between related tables and ensure referential integrity.
- **NOT NULL Constraints**
Important fields such as patient names and appointment dates cannot be left empty.
- **UNIQUE Constraints**
Fields such as employee numbers and email addresses must remain unique.
- **CHECK Constraints**
The system will enforce rules such as ensuring that payment amounts cannot be negative and appointment status values remain valid.
- **Default Constraints**
Default values such as “Pending” for appointment status will improve efficiency and consistency.

3.3.4 Conclusion

The proposed Hospital Management System requires accurate, secure, and reliable data management processes. Proper data analysis, integrity mechanisms, and database constraints will improve efficiency, reduce errors, and support effective healthcare service delivery.

3.4 Weakness of the Current System

One big weakness is fragmentation. Lots of public hospitals run on isolated paper documents, handwritten logs, and disconnected digital tools. That leads to inactive check-ins, repeated entries, lost paperwork, spotty tracking, plus shaky data summaries. Studies into electronic health records in South Africa show change crawls forward - systems fail to link up properly, trained IT workers are too few, leaders hesitate, buying equipment hits roadblocks, while money for tech improvement stays low.

Out there in Limpopo, delays show up right away when people wait - records get tangled too. The 2024 check on clinics spotted a typical wait lasting three hours and four minutes; nearly two out of five patients called that too long. When voices rose, most pointed at missing workers, while some tagged inactive routines instead, and others named jumbled files or documents gone sideways.

Most posts sit empty. The 2024/25 yearly summary for Limpopo reveals more than half of planned roles unfilled - especially doctors, nurses, medics, and experts. Because of that, waiting lines grow

longer, paperwork slows down, tracking patients gets messy, records fall behind, and care slips. When staff numbers stay low, everything else wobbles.

Bad roads and weak systems add stress. In Limpopo, one out of every three clinics was falling apart, most lacked room to grow, almost half had broken toilets, while just over a quarter could count on backup power. Think about it - hospitals cannot run without steady electricity, open floors, machines that work, shelves for records, or live internet lines.

Limpopo's efforts show movement forward - every one of its 37 hospitals now runs both Electronic Data Interchange and Patient Verification Systems. These tools help manage fees, sort patient categories, and cut down on paperwork. Yet despite that step, none operate under a single connected system combining admissions, medical notes, prescriptions, test outcomes, invoicing, bookings, transfers, records tracking, and summaries of care. The pieces remain apart.

So here's why the HMS project makes sense:

Hospitals already use some digital tools, yet they don't connect well, leading to slow workflows and repeated tasks. Missing links between systems make it hard to follow patients accurately through care stages. Workers end up overloaded because of these gaps. Decisions suffer when information isn't available right away. The new setup needs to handle how people sign in at facilities. It must store health histories electronically. Scheduling visits should be part of it, too. Charging for services must be built in. Tracking medical from pharmacy logs matters just as much. Who does what as team members ought to be clear within the software. Generating activity-based summaries will help with oversight. Accessing files safely, only by approved users, forms a core piece.

3.5 Analysis of the Proposed System (Functional Requirements)

The section will detail the full functional requirements for the HMS.

3.5.1 FR-01: Patient Data Management

Replaces paper-based files for patient registration and record management with digital patient profiles and records management that accessible to both Hospital Admin/Nurse and Patient directory

Detailed Workflow for Admin/Nurse Registers Patient

See sequence diagram below

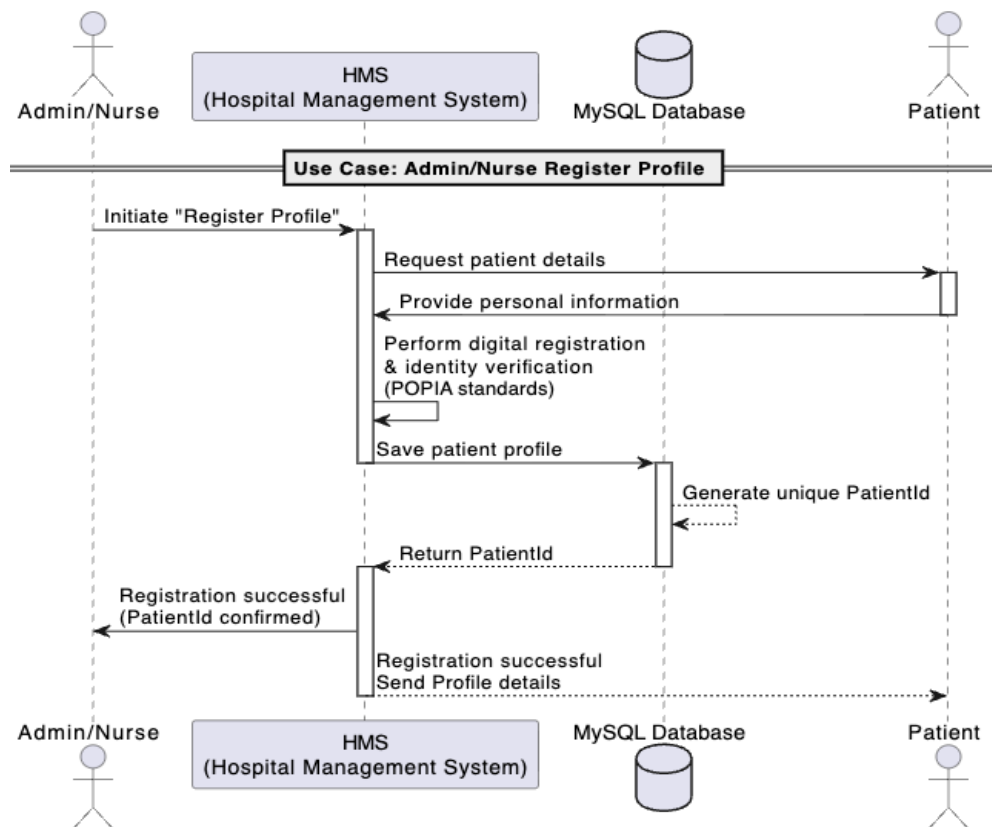
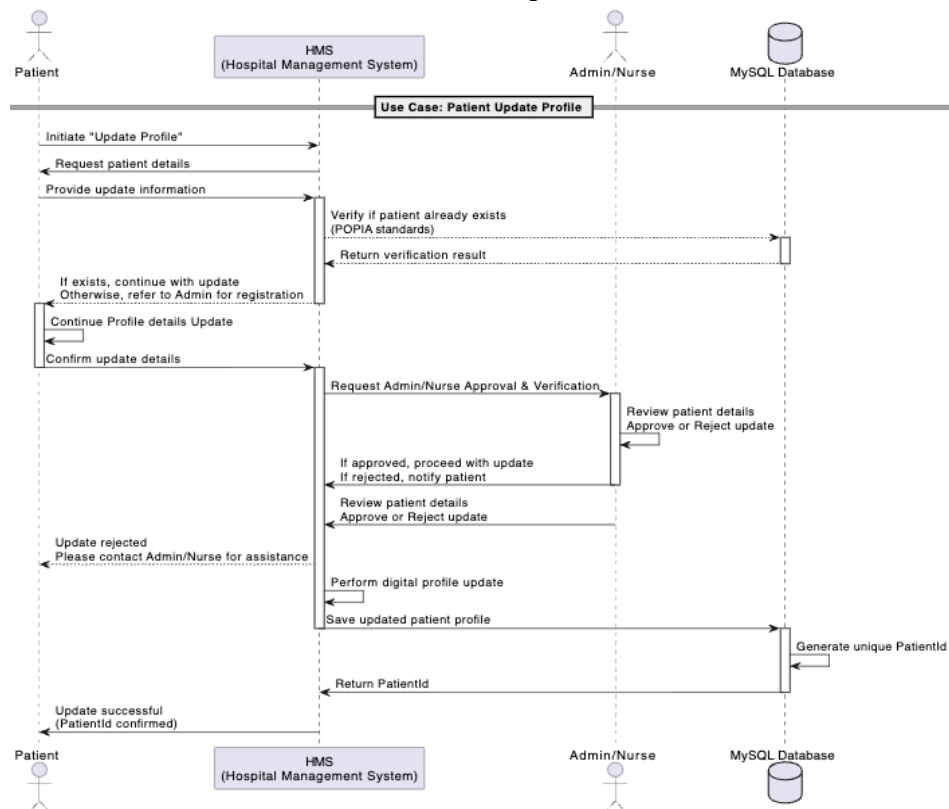


Figure 0-1: FR-01 Sequence Diagram – Admin

Detailed Workflow for Patient Details Update



*Figure 0-2: FR-01 Sequence Diagram – Patient Update Record***Data Stored**

Field Name	Data Type	Value
PatientId	INT (PK)	Unique auto-increment identifier
PatientName	VARCHAR(255)	Full name of the patient
PatientIdNumber	VARCHAR(13)	Unique South African ID number
PatientContact	VARCHAR(20)	Mobile or email contact details
PatientMedicalNotes	LONGTEXT	Encrypted clinical history and intake notes

*Table 3: Patient Profile***Business Rules**

1. Patient ID Number must be a unique 13-digit SA ID.
2. Field Validations enforced as per DB data type
3. Access is restricted to authorized medical staff to replace paper-based files.

3.5.2 FR-02: Medicine/Prescriptions Issuance

Digital tracking system of prescriptions and dispensing medicines as regulated. A history of issued prescriptions will be saved.

Detailed Workflow for Issuance of Digital Prescription

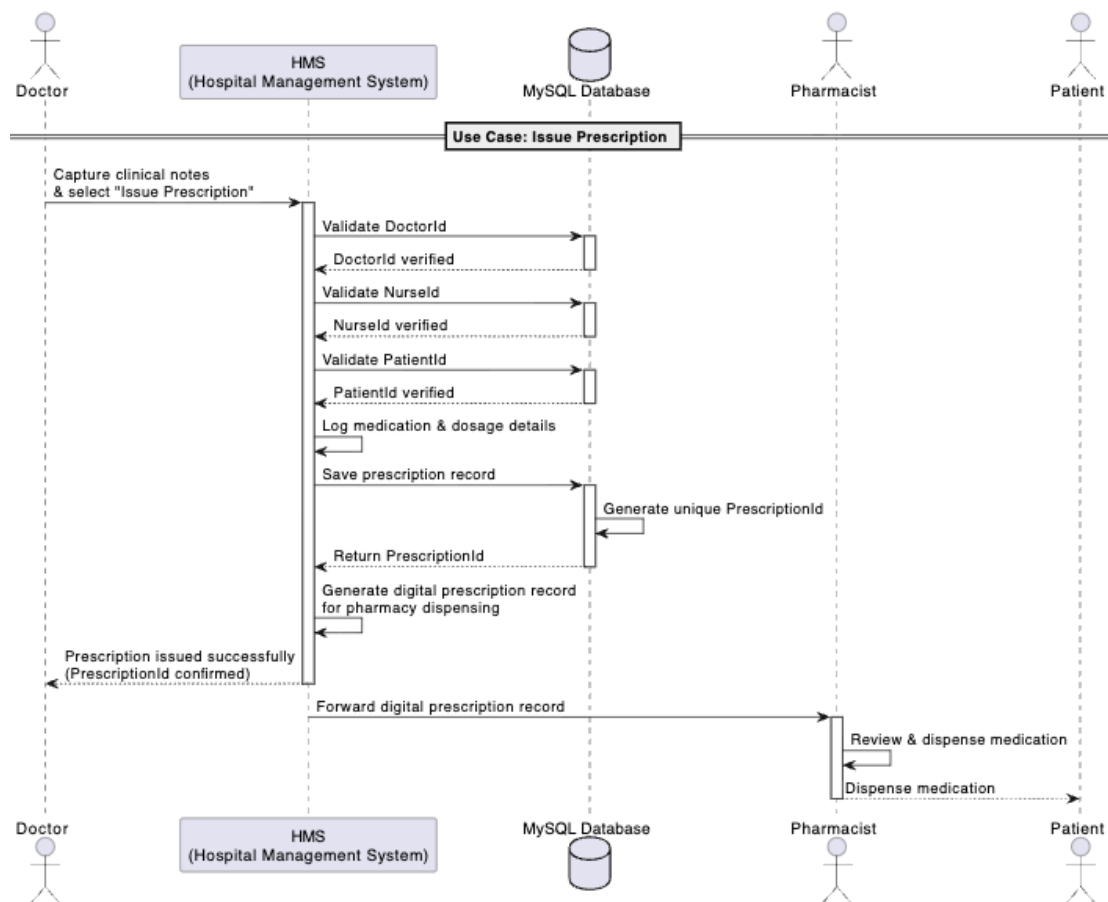


Figure 0-3: FR-02 Sequence Diagram

Data Stored

Field Name	Data Type	Value
RxId	INT (PK)	Unique prescription identifier
PatientId	INT (FK)	Reference to the patient record
DoctorId	INT (FK)	Reference to the issuing doctor
NurseID	INT (FK)	Reference to the attending nurse
Medication	VARCHAR(255)	Name of the prescribed medicine
Dosage	VARCHAR(100)	Frequency and strength instructions
Quantity	INT	Total units dispensed

Table 4: Prescriptions Data Store

Business Rules

1. Doctor, Attending Nurse and Patient must be verified by the system
2. Audit log on all issued prescriptions must be logged against patient and doctor profile

3.5.3 FR-03: Appointment Scheduling

For managing patient and doctor appointments and allow patients to request for appointment rescheduling, with the attending doctor having to approve or reject.

Detailed Workflow for Appointments: Patient/Doctor Visits

See sequence diagram below

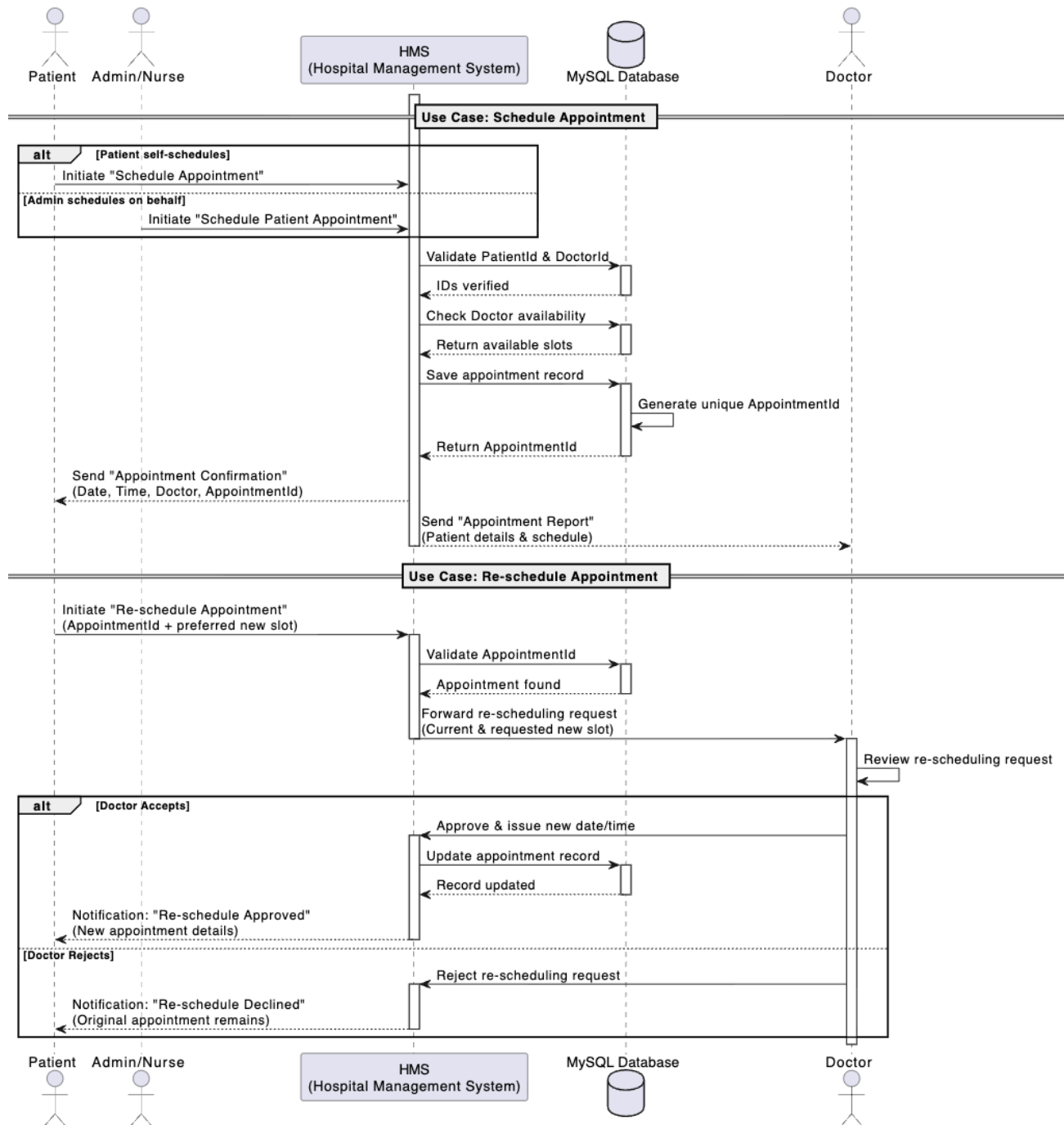


Figure 0-4: FR-03 Sequence Diagram

Data Stored

Field Name	Data Type	Value
ApptId	INT (PK)	Unique appointment identifier
PatientId	INT (FK)	Reference to the patient
DoctorId	INT (FK)	Reference to the doctor
Date	DATETIME	Scheduled date and time slot
Comments	VARCHAR(255)	Reference to patient/doctor comments on appointments
Status	VARCHAR(20)	Scheduled, Completed, or Cancelled

Table 5: Appointments

Business Rules

1. Ensure that one doctor attends to multiple scheduled patients (One-to-Many).
2. Patient cannot amend appointment scheduled
3. Patient can request for appoint change and only attending doctor can approve or decline

3.5.4 FR-04: Billing Management

Manages patient billing record for issues patient billing invoice and statements. The system will also generate itemized in-patient billing report and summary of services. At payment process, Patient will be offered various payment options with an account plan of up-to 12 months to pay with no interest (Government Subsidy)

Detailed Workflow for Patient Billing System

Generate In-Patient Billing Report

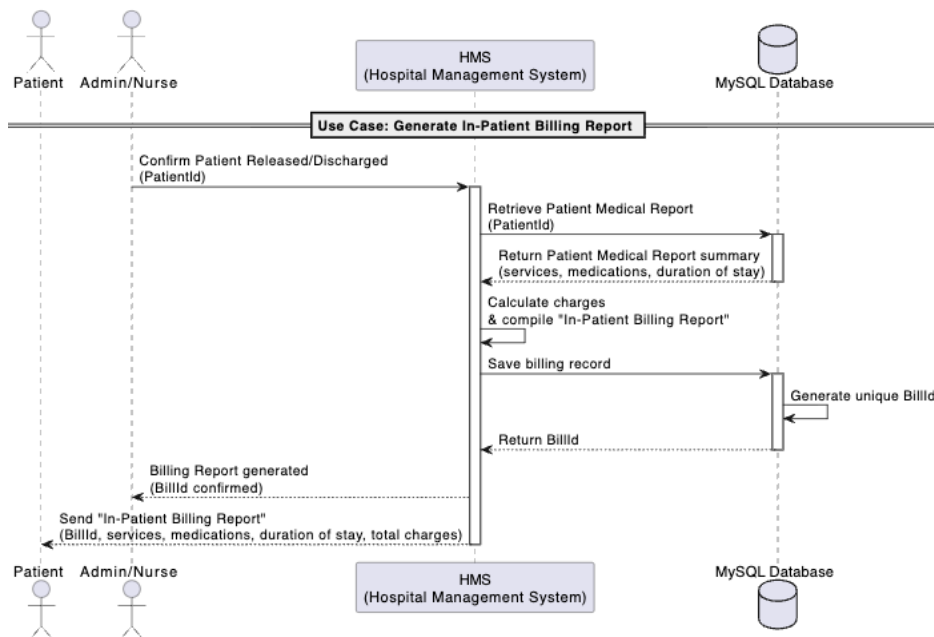


Figure 0-5: In-Patient Billing Report

Generate Invoice

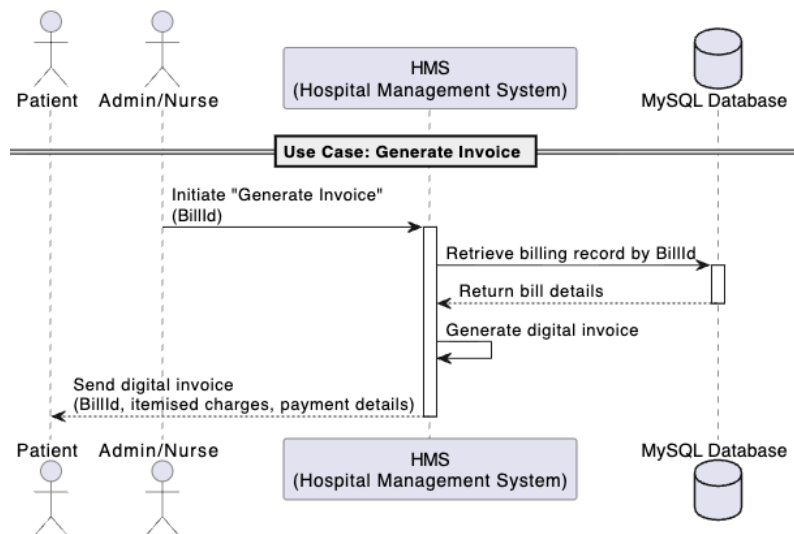


Figure 0-6: Generate Invoice

Select Payment Method

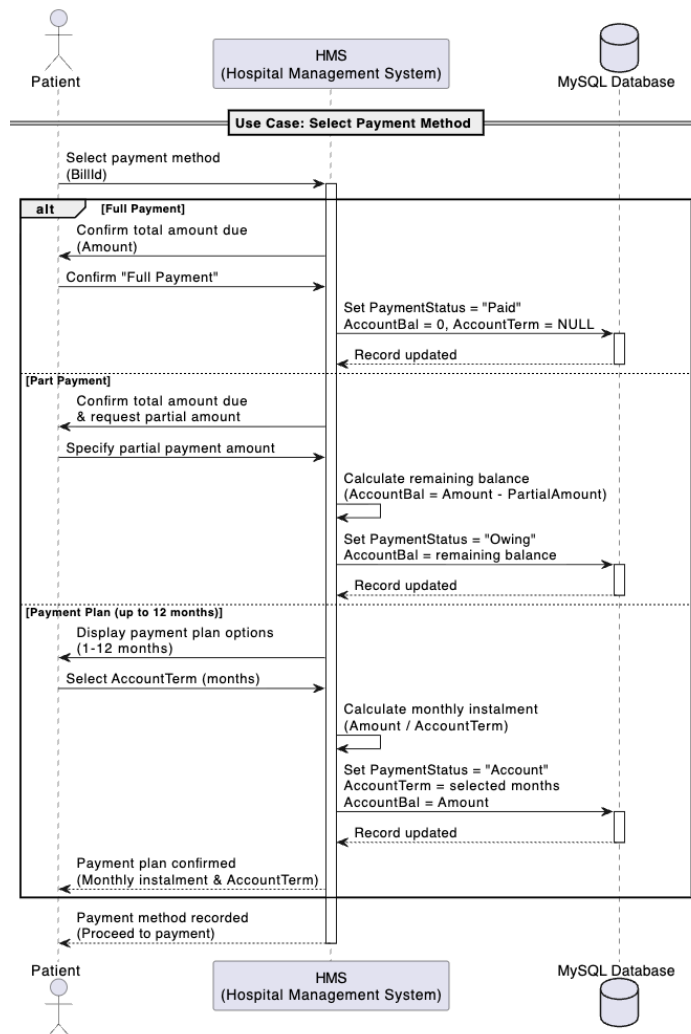


Figure 0-7: Payment Methods

Process Payment

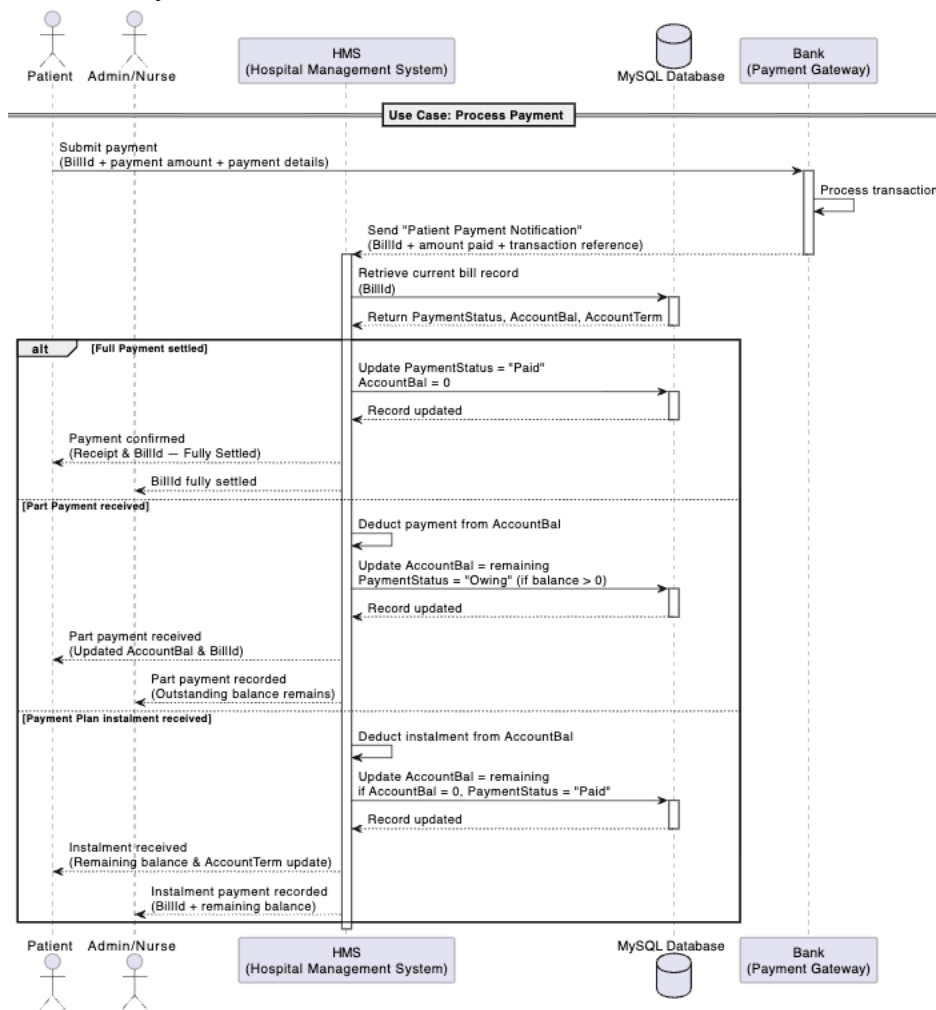


Figure 0-8: Process Payment

Data Stored

Field Name	Data Type	Value
BillId	INT (PK)	Unique billing record identifier
PatientId	INT (FK)	Reference to the patient
Amount	DOUBLE(10,2)	Total financial charges
PaymentStatus	VARCHAR(20)	Paid, Owing or Account (payment plan)
AccountBal	DOUBLE(10,2)	Reference account balance owing from total amount
AccountTerm	INT	Terms of Account payment plan
Date	DATE	Invoice issuance date

Table 6: Billing Data

Business Rules

1. System must support various billing models (Cash/Medical Aid/Cards).
2. Patient Billing Record cannot be deleted until Payment Status is "Paid".

3.5.5 FR-05: Risk Management Tracking

Risk tracking management function to avoid operational issues on inventory and human resources in the hospital's day-to-day running.

- Inventory: An early warning tracking system to manage stock and reordering
- HR: manage doctor/nurse shift and staffing resources

Detailed Workflow for Inventory Management

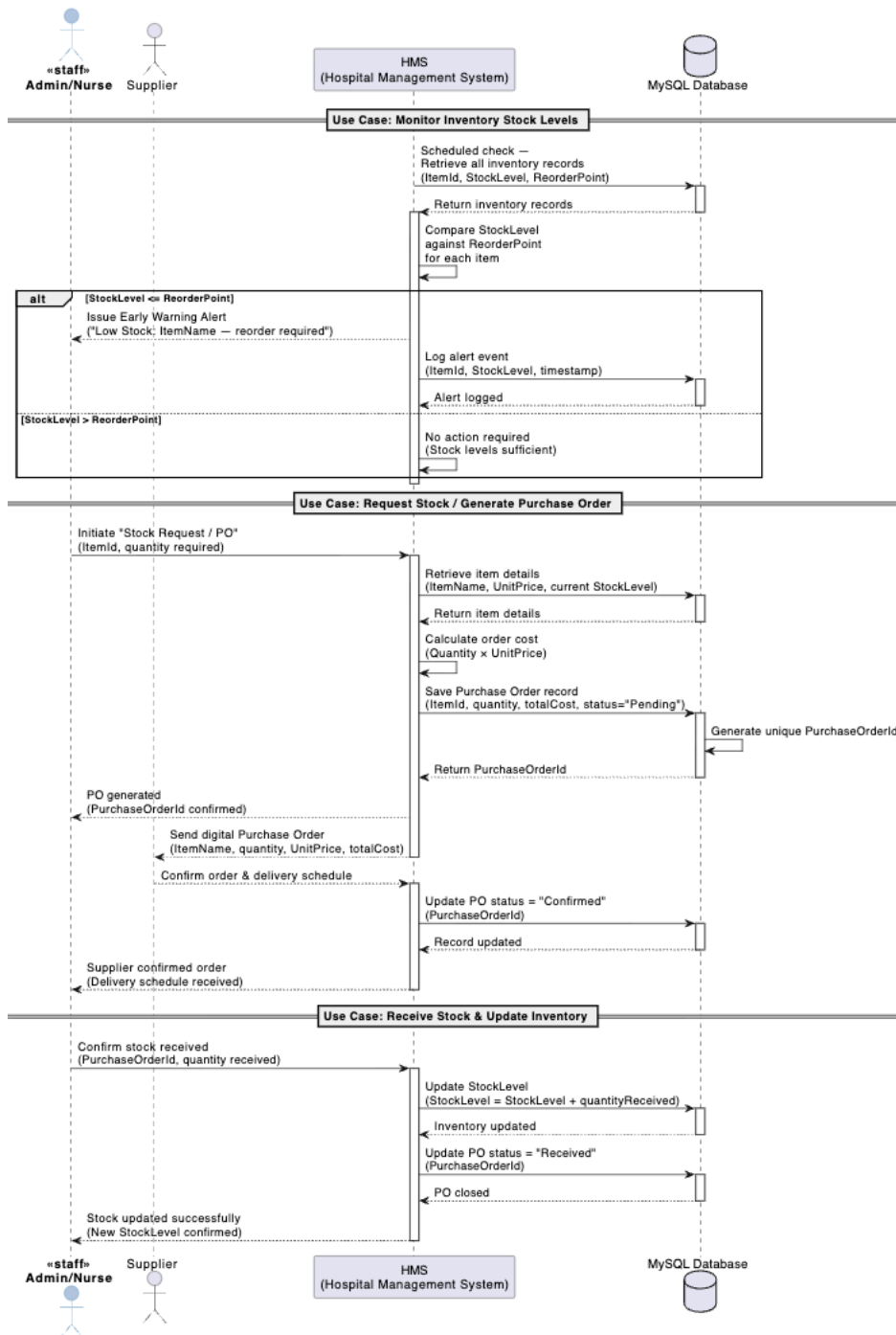


Figure 0-9: FR-05 Sequence Diagram – Inventory Management

Detailed Workflow for HR (Staff) Shift Management

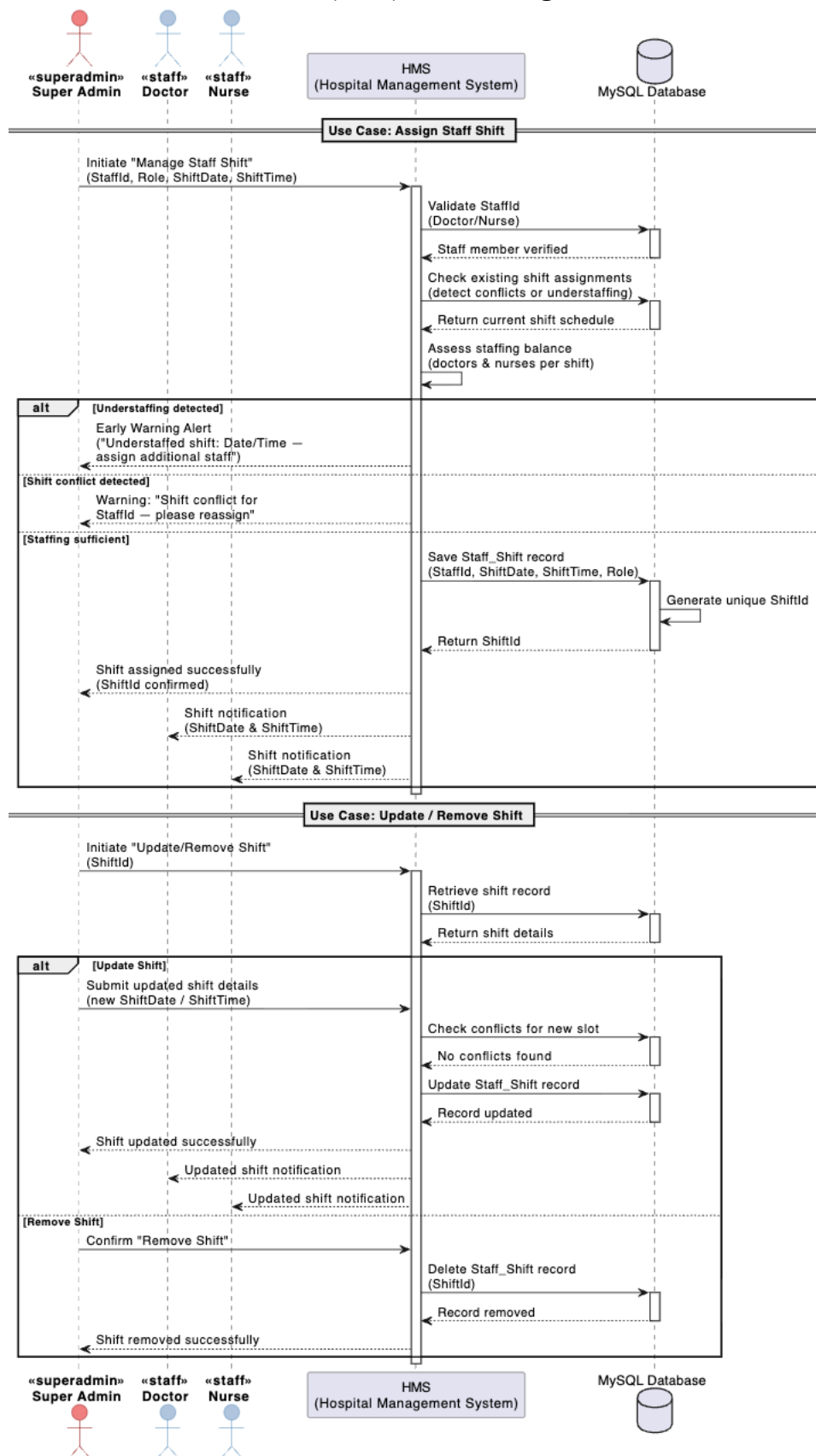


Figure 0-10: FR-05 Staff Shift Management

Data Stored

Inventory

Field Name	Data Type	Value
ItemId	INT (PK)	Unique inventory item identifier
ItemName	VARCHAR(255)	Name of medicine or supply
StockLevel	INT	Current quantity on hand
ReorderPoint	INT	Minimum threshold for alert
UnitPrice	DOUBLE(10,2)	Cost per individual unit
AlertId	INT (PK)	Unique alert identifier
AlertDate	DATETIME	Timestamp alert was triggered

Table 7: Inventory Data

Supplier

Field Name	Data Type	Value
SupplierId	INT (PK)	Unique supplier identifier
SupplierName	VARCHAR(255)	Supplier company name
ContactEmail	VARCHAR(255)	Supplier email for PO delivery
Phone	VARCHAR(20)	Supplier contact number

Table 8: Supplier Data

Purchase Order

Field Name	Data Type	Value
PurchaseOrderId	INT (PK)	Unique purchase order identifier
ItemId	INT (FK)	Reference to inventory item
SupplierId	INT (FK)	Reference to supplier
Quantity	INT	Units ordered
TotalCost	DOUBLE(10,2)	Quantity $\times$ UnitPrice
Status	VARCHAR(20)	Pending, Confirmed, or Received
OrderDate	DATE	Date PO was raised

Table 9: Purchase Order Data

Stock Receipt

Field Name	Data Type	Value
ReceiptId	INT (PK)	Unique receipt identifier
PurchaseOrderId	INT (FK)	Reference to purchase order
QuantityReceived	INT	Actual units delivered
ReceivedDate	DATE	Date stock was received

Table 10: Stock Receipt Data

Staff

Field Name	Data Type	Value
StaffId	INT (PK)	Composite key identifier (i.e. identifies staff member)
DoctorId	INT (FK, nullable)	References Doctor->DoctorId
NurseId	INT (FK, nullable)	References Nurse->NurseId
Role	VARCHAR(20)	Doctor or Nurse
Status	VARCHAR(20)	Active or Inactive

Business Rules

1. Automatic triggers for stock reordering when StockLevel drops below ReorderPoint.
2. A PO (purchase order) can only be issued to one Supplier
3. A supplier can be issued multiple separately POs
4. Only a Super Admin user can assign/update/remove shifts allocated to Staff

3.5.6 FR-06: Doctor/Nurse Management

To manage doctor and nurse profiles including other admin staff. A super Admin user with high-level CRUD access will be used to add/update/remove users on the system for security reasons.

Detailed Workflow for Doctor/Nurse Profile Management

See diagram below

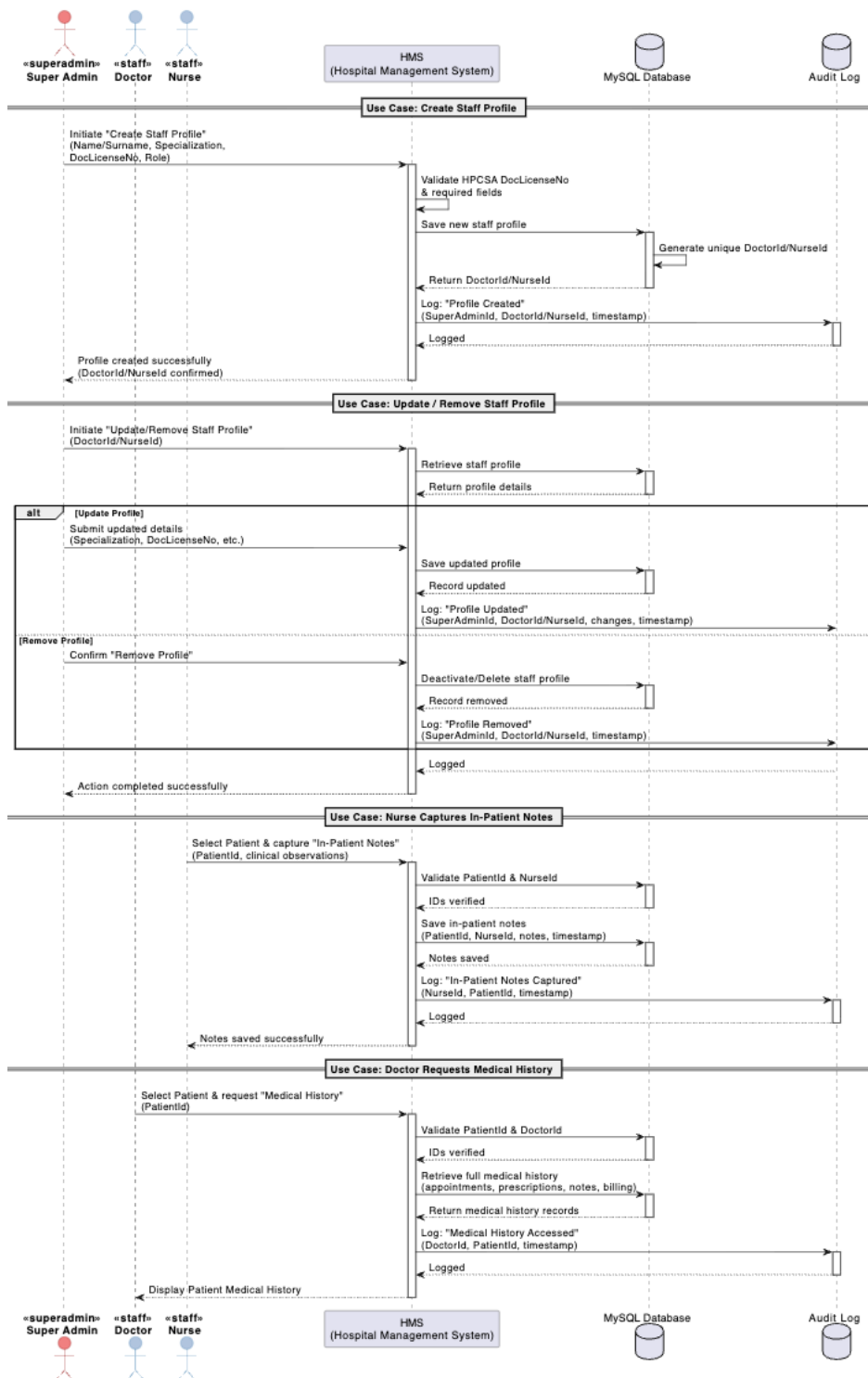


Figure 0-11: FR-06 Sequence Diagram

Data Stored

Field Name	Data Type	Value
DoctorId/NurseId	INT (PK)	Unique internal staff identifier
Full_Names	VARCHAR(255)	Staff member's full legal name
Specialization	VARCHAR(100)	Department or medical specialty
DocLicenseNo	VARCHAR(50)	HPCSA regulatory license number

Table 11: FR-06 Data

Business Rules

- All profiles must be validated and secured;
- clinical notes cannot be edited once saved to maintain an audit trail.
- Only super Admin can remove nurse/doctor

3.5 Non-Functional Requirements

Analysis of the Non-Functional Requirements of the HMS referencing the functional requirements in how the two are linked

ID	Scope	DB Relationships	Requirement Description
NFR-1	Security	Audit_Log	Compliance with POPIA/NHA; 2FA and encryption of PatientMedicalNotes.
NFR-2	Performance	All Tables	Data entry and reporting for ApptId and BillId must process in faster response times.
NFR-3	Reliability	Offline-First - SyncLog	Uses SyncStatus to maintain core entry during internet/power downtime.
NFR-4	Interop	API Gateway	Uses secure APIs for sharing Medication data with external healthcare ecosystems.
NFR-5	Scalability	INT-64bit IDs	Architecture supports vertical and horizontal growth for 3rd party integrations.
NFR-6	Auditability	Audit_Log	Secure logging of every UserId action and Timestamp for a full audit trail.

Table 12: NFR Analysis Map

3.6.1 Reliability & Sync Logic (NFR-3)

This is to ensure the **Hybrid Cloud-Edge** model works by tracking data states during connectivity outages to offer the Offline-first reliability feature.

Data Stored

Field Name	Data Type	Value
LogId	INT (PK)	Unique identifier for each audit entry
TableName	VARCHAR(50)	Identifies table with changes to be updated
RecordId	INT	Reference cached record with the specific change in the table
SyncStatus	VARCHAR(10)	'Local' (cached) or 'Synced' (uploaded to cloud DB)
Timestamp	TIMESTAMP	Exact time of local activity capture
IP_Address	VARCHAR(45)	Network ID of the handheld device used during offline mode

Table 13: Offline Sync Data

3.6 Data Modelling for Proposed System

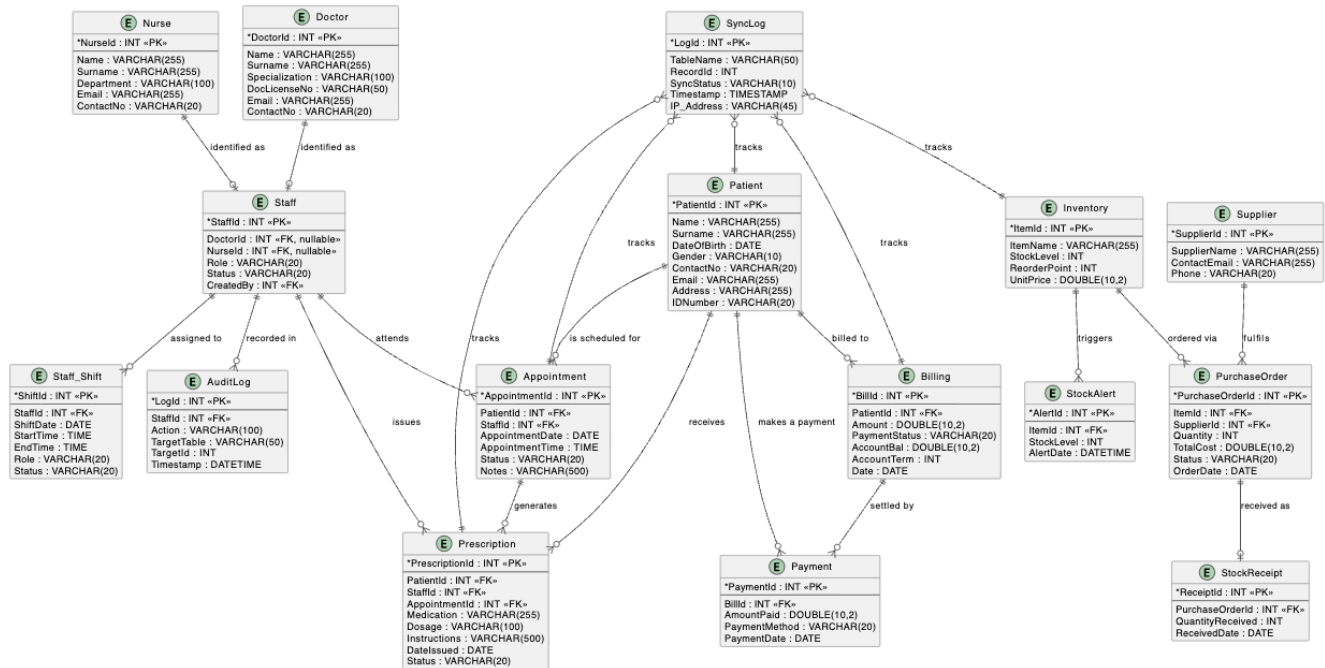


Figure 6-1: HMS ERD

The Entity Relationship Diagram (ERD) represents the structural design of the Hospital Management System database. It illustrates how core entities such as Patient, Doctor, Nurse, and Staff interact with operational entities including Appointment, Prescription, Billing, Inventory, and AuditLog, with Offline SyncLog Mode.

The system follows a relational design where entities are connected using primary and foreign keys to ensure data consistency and reduce redundancy. The schema is divided into core entities and operational entities to improve clarity, scalability, reliability, and maintainability.

The ERD is designed to:

- Represent real-world hospital workflows in a structured relational format
- Define clear relationships between patients, doctors, and hospital operations
- Ensure data integrity through primary and foreign key constraints
- Support efficient and scalable database management

Key Design Characteristics

- One-to-Many relationships are used extensively to support scalability
- Separation of core entities and operational entities improves system clarity
- Referential integrity is enforced through foreign key constraints
- Transactional entities support automated processes such as prescriptions and billing updates

The ERD enforces key hospital business rules, including

- Preventing overlapping appointments for patients and doctors
- Ensuring prescriptions are linked to valid patients and doctors
- Validating medication existence before prescription issuance
- Maintaining non-negative inventory stock levels
- Ensuring billing values remain valid and consistent

Data integrity is maintained through

- Primary key uniqueness for entity identification
- Foreign key constraints for relational consistency
- Normalized table design to reduce redundancy
- Controlled transactional processes for critical operations
- A Super Admin user with all CRUD rights to manage top-level relationships and functions

The database design

- Accurately represent real hospital workflows
- Ensure scalability for future system growth
- Maintain high data integrity and consistency
- Support secure and efficient data operations across all modules

The following section summarizes the key relationships defined in the ERD and explains how they enforce business rules and maintain data integrity.

Parent Entity	Child Entity	Relationship Type	Foreign Key in Child	Description
Nurse	Staff	1:N	NurseId	Nurse entities are associated with staff records for role management and workforce tracking.
Doctor	Staff	1:N	DoctorId	Doctor entities are linked to staff records to support generalized staff administration.
Staff	Staff_Shift	1:N	StaffId	Each staff member can be assigned multiple work shifts over time.
Staff	AuditLog	1:N	StaffId	Each staff member can generate multiple audit log records

Parent Entity	Child Entity	Relationship Type	Foreign Key in Child	Description
				through system activity.
Patient	Appointment	1:N	PatientId	A patient can schedule multiple appointments for consultations or treatments.
Doctor	Appointment	1:N	DoctorId	A doctor can manage multiple patient appointments.
Patient	Prescription	1:N	PatientId	A patient may receive multiple prescriptions during different visits or treatments.
Doctor	Prescription	1:N	DoctorId	A doctor can issue multiple prescriptions to different patients.
Patient	Billing	1:N	PatientId	A patient can have multiple billing records related to medical services or treatments.
Inventory	StockAlert	1:N	ItemId	Inventory items can generate multiple low-stock alerts when stock thresholds are reached.
Inventory	PurchaseOrder	1:N	ItemId	An inventory item can appear in multiple purchase orders for replenishment purposes.
Supplier	PurchaseOrder	1:N	SupplierId	A supplier can fulfill multiple purchase orders from the hospital.
PurchaseOrder	StockReceipt	1:N	PurchaseOrderId	A purchase order may result in one or more stock receipt records upon delivery.

Parent Entity	Child Entity	Relationship Type	Foreign Key in Child	Description
Patient	SyncLog	1:N	Relationship implemented through linked staff role structure	SyncLog is logically associated with patient-related system activity for synchronization tracking.

Table 14: HMS Database Relationships

3.7.1 Considerations

The database is designed to the Third Normal Form (3NF) standard to eliminate redundancy, ensure dependency consistency, and improve data integrity across all entities.

- Core vs operational separation improves scalability and maintainability
- Use of composite key for Staff Personnel identifying either Doctor or Nurse
- One-to-many relationships reflect real hospital workflows
- Normalized structure reduces duplication in prescriptions, billing, and inventory
- PatientId and DoctorId must be valid foreign keys in Appointment
- Stock level must always be ≥ 0
- Billing amount must be > 0
- Appointment times must not overlap for same doctor or patient
- Medication must exist before prescription is created

3.7.2 Integrity enforcement

- Primary keys ensure unique identification
- Foreign keys enforce relational consistency
- Triggers enforce stock and prescription rules
- Transactions prevent partial updates in critical operations

The design supports scalability through separation of transactional and master data, allowing efficient handling of increasing hospital records over time.

3.7.3 Assumptions

- Each appointment is assumed to last a single time slot
- A prescription must contain at least one medication
- Staff roles are assumed to be predefined in the system

Overall, the database design prioritizes data integrity, scalability, and real-world applicability through structured relationships, enforced constraints, and normalized entity design.

4 System Phase

4.1 Introduction

The Analysis Phase is a critical stage in the development of the Hospital Management System. During this phase, the current processes, user requirements, and operational challenges within the hospital environment are examined and documented. The purpose of the analysis phase is to gather accurate information about how the hospital operates, identify existing problems, and determine the requirements needed for the proposed system. The information collected serves as the foundation for designing a system that will improve efficiency, accuracy, and service delivery within the hospital.

4.2 System Design

To obtain a clear understanding of the hospital's operations and requirements, several information-gathering techniques were used. These methods include observation, participatory observation, and interviews.

Observation involves watching hospital staff perform their daily tasks within different departments. Through observation, the analyst can understand how patient information is recorded, how appointments are scheduled, how medical records are managed, and how billing processes should be conducted.

The observation method helps identify existing workflows and procedures, delays and bottlenecks in service delivery, manual processes that can be automated, and communication patterns between departments. By monitoring activities in various hospital departments such as reception, consultation rooms, pharmacies, laboratories, and billing offices, important insights into the hospital's operations can be obtained.

Participatory observation involves actively engaging with hospital staff while they perform their duties. The analyst may assist or follow employees through their daily activities to gain a deeper understanding of their responsibilities and challenges.

This method helps the analyst understand user experiences firsthand, identify practical difficulties faced by staff, discover system requirements that may not be apparent through observation alone, and gain insight into interactions between different departments. For example, participating in patient registration procedures permits the analyst to understand the information required during admission and the challenges associated with maintaining patient records manually.

Interviews involve direct discussions with stakeholders such as hospital administrators, doctors, nurses, receptionists, pharmacists, laboratory technicians, and patients. Interviews provide detailed information about user needs, expectations, and concerns regarding the proposed Hospital Management System.

The interview process helps gather information about current system limitations, functional requirements of the new system, security and confidentiality requirements for patient records, reporting and management needs, and suggestions for improving hospital operations. Both structured and semi-structured interviews can be conducted to ensure that all relevant information is collected while allowing participants to provide additional insights.

The use of observation, participatory observation, and interviews provides comprehensive information about the hospital's operational processes and stakeholder requirements. The data collected during the analysis phase forms the basis for designing an effective Hospital Management System that enhances patient care, improves administrative efficiency, and supports decision-making within the hospital.

4.3 Architectural Design

The architectural design of the HMS is key critical in the software engineering design process as it outlines the link between the overall design of the HMS and its requirements engineering, further highlighting the main structural components of the system

4.3.1 Software Architectural Design

The HMS will follow a layered architectural design (reference) in a 4-Tier Architecture Design, with the extra layer for the offline/cloud component that is key to the system. The design is to separate issues, improve maintainability, easier to scale and support the offline-first hybrid cloud model.

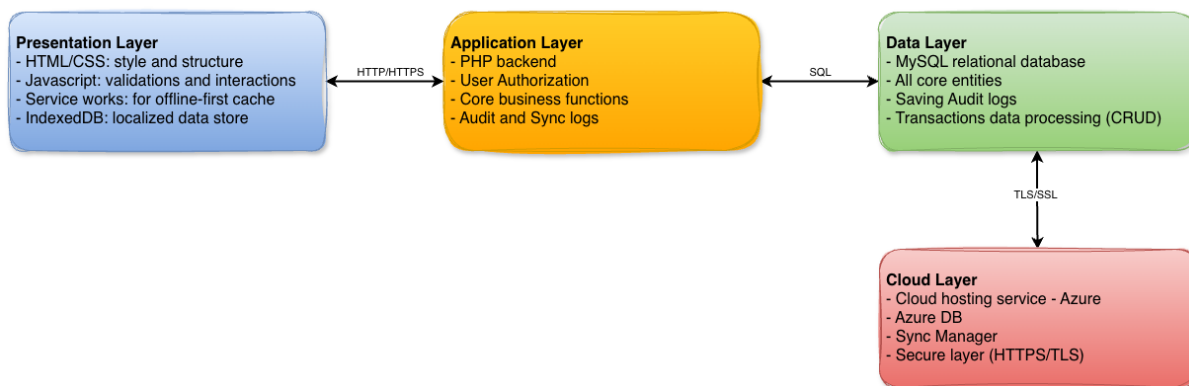


Figure 2: Software Architecture Design

Layer	Language/Technology	Roles
Presentation	HTML, CSS, JavaScript, Service Worker, IndexedDB	Front-end UI, offline caching, local storage
Application	PHP	Business logic, Functional Requirements modules, audit logging
Data	MySQL Server	Relational data database, constraints, triggers, enties
Cloud / Sync	Azure SQL, SyncLog manager	Cloud replica, offline sync

Table 15: Software Architecture Layers

4.3.2 Hardware Architectural Design

The HMS hardware infrastructure is organised across four areas, designed to provide reliable and efficient operations in the day-to-day running of a hospital, including offline-first continuity.

Focus Areas	Device Types/Components	Purpose
Client devices	PCs, laptops, tablets, mobiles	Browser-based HMS access
Hospital LAN	Firewall, switches, Wi-Fi APs, UPS	Internal routing and resilience
On-premise servers	Web server, MySQL server, backup server	Core local processing
Azure cloud	Azure SQL, Sync service, API gateway, Blob storage	Cloud sync and integrations

Table 16: Hardware Design Areas

4.3.3 Network Architectural Design

The HMS network is designed around two distinct traffic flows; the online and offline, ensuring continuity of hospital operations regardless of connectivity status. All transaction data is encrypted using HTTPS/TLS 1.2/1.3.

- **Online traffic flow**
When connected, client devices communicate via the hospital Wi-Fi or wired LAN, passing through the perimeter firewall before reaching the web server. All traffic between the client, web server, and MySQL database is encrypted using HTTPS/TLS. When internet is available, the SyncLog manager pushes any queued data changes to the Azure cloud replica via a TLS-secured tunnel.
- **Offline traffic flow**
When connectivity is lost, client devices automatically switch to offline mode. The Service Worker intercepts all browser requests and serves cached responses, while IndexedDB stores any new data entries locally. All changes are logged in the SyncLog table with a status of "Local". Once connectivity is restored, the SyncLog manager reconciles all queued records against the Azure SQL cloud database, updating each entry to "Synced".
- **Security across the network**
All communication traverses HTTPS/TLS 1.2/1.3. The perimeter firewall governs all inbound and outbound traffic. Role-based access control and two-factor authentication (2FA) as an option, as defined in NFR-1, are enforced at the application layer before any data is served, regardless of whether the request originates from the LAN or the cloud.
- **External integrations**
The Azure API gateway manages all connections to external healthcare systems, enabling POPIA-compliant, needs-basis sharing of patient data with third-party systems such as laboratories, insurance providers, or other hospitals. All external API calls are authenticated and logged to the Audit Log (NFR-6).

Protocol / Component	Purpose	Requirements	Reference
HTTPS / TLS 1.3	All data in transit encrypted	NFR-1 , NFR-3	
Perimeter firewall	Filters all inbound/outbound traffic	NFR-1	
Service Worker	Intercepts requests during offline mode	NFR-3	
IndexedDB	Local browser data store for offline entries	NFR-3	
SyncLog manager	Queues and reconciles offline changes to cloud	NFR-3	
Azure API gateway	Manages external healthcare integrations	NFR-4	

Table 17: Network Architecture

4.3.4 Class Diagram

The HMS class diagram is structured around three logical groupings:

- **Core people entities:** Patient, Doctor, Nurse, and Staff form the human-facing core. Doctor and Nurse both link into Staff via a one-to-one relationship, allowing the system to manage all personnel through a unified staff record. Staff then links to StaffShift for scheduling and AuditLog for activity tracking.
- **Operational entities:** Appointment, Prescription, and Billing are the transaction-heavy classes, each linked back to Patient and Doctor via foreign key relationships.
- **Inventory and supply chain entities:** Inventory, Supplier, PurchaseOrder, and StockReceipt manage the stock risk tracking system (FR-05). The triggerAlert() method on Inventory fires automatically when stockLevel drops below reorderPoint, implementing the early warning system.
- **Supporting entities:** AuditLog captures every system action for compliance (NFR-1, NFR-6), while SyncLog tracks all offline data changes for the hybrid cloud-edge model (NFR-3).

Class	Key attributes	Key methods
Patient	patientId, name, idNumber, medicalNotes	register(), updateProfile(), viewHistory()
Doctor	doctorId, specialization, licenseNo	issuePrescription(), approveAppointment()
Nurse	nurseId, department, licenseNo	captureNotes(), admitPatient(), dischargePatient()
Staff	staffId, doctorId/nurseId, role, status	activate(), deactivate()
StaffShift	shiftId, staffId, shiftDate, startTime, endTime	assignShift(), removeShift()
Appointment	apptId, patientId, doctorId, date, status	schedule(), requestReschedule(), cancel()
Prescription	rxId, patientId, doctorId, medication, dosage	issue(), dispense()
Billing	billId, patientId, amount, paymentStatus, accountBal	generateInvoice(), processPayment(), updateBalance()
Inventory	itemId, stockLevel, reorderPoint, unitPrice	checkStock(), triggerAlert(), updateStock()
Supplier	supplierId, supplierName, contactEmail	receivePO(), confirmOrder()
PurchaseOrder	purchaseOrderId, itemId, supplierId, quantity	createPO(), confirmPO()

Class	Key attributes	Key methods
StockReceipt	receiptId, purchaseOrderId, quantityReceived	receiveStock()
AuditLog	logId, staffId, action, timestamp, ipAddress	logAction(), retrieveLog()
SyncLog	logId, tableName, recordId, syncStatus, timestamp	logLocal(), syncToCloud()

Table 18: HMS Class Diagram

4.4 Physical Design

The HMS physical architecture is structured across four tiers, each serving a distinct role in the system's operation.

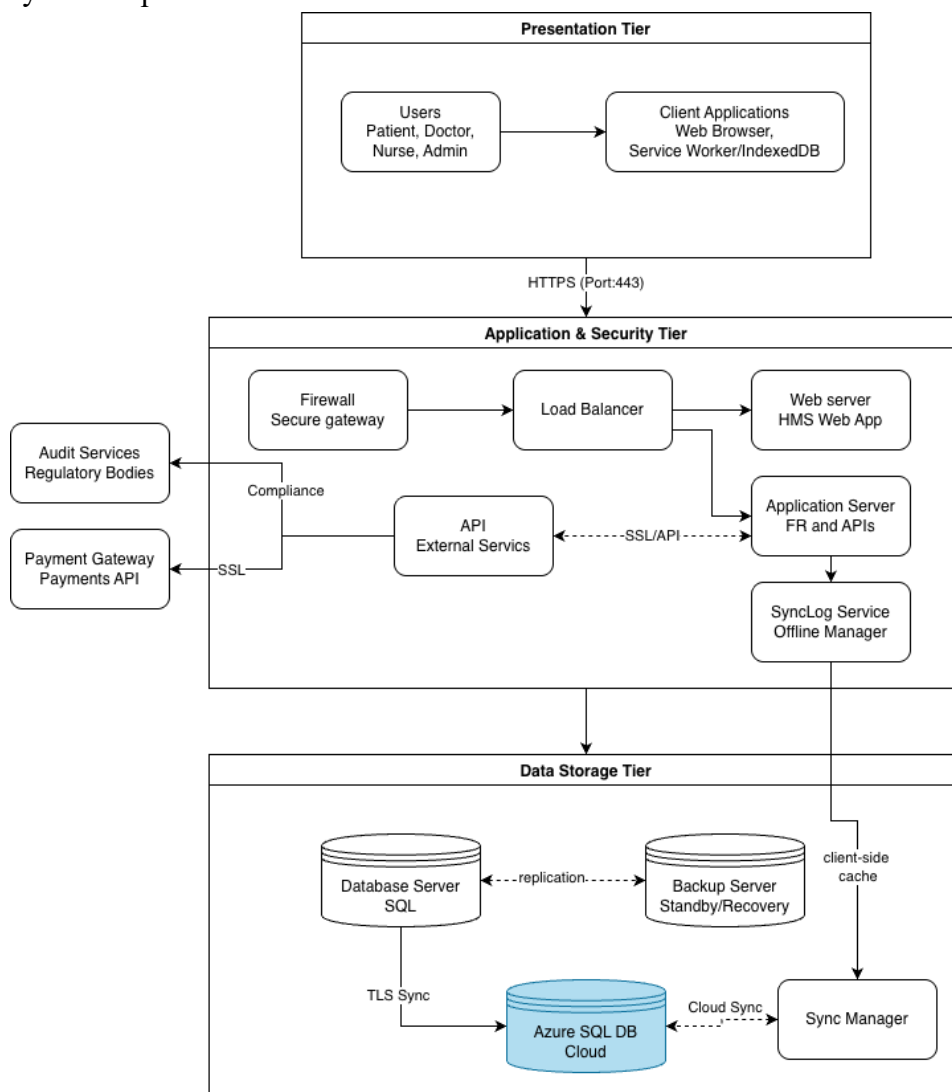


Figure 3: HMS Physical Design

4.5 Database Design

The Hospital Management System (HMS) uses a relational database implemented in MySQL. All tables conform to Third Normal Form (3NF), eliminating redundancy, ensuring each non-key attribute depends only on the primary key, and enforcing referential integrity through foreign key constraints.

4.5.1 Physical Design

The following principles govern the physical database design:

- **Primary keys:** Every table uses an INT AUTO_INCREMENT primary key for efficient indexing and join performance.
- **Foreign keys:** All relationships from the ERD are enforced via FOREIGN KEY constraints to prevent orphaned records.
- **CHECK constraints:** Business rules such as non-negative stock levels, positive billing amounts, and valid status values are enforced at the database level.
- **DEFAULT values:** Common fields such as PaymentStatus, SyncStatus, and Timestamps use defaults to reduce application-layer overhead.
- **Indexes:** Indexes are placed on frequently queried foreign key columns and date fields to optimize query performance.
- **Triggers:** Five database triggers automate key business rules — audit logging, stock updates, billing status, and SyncLog defaults.

4.5.2 Third Normal Form (3NF) Compliance

The HMS database schema satisfies Third Normal Form across all tables:

- **1NF** - All tables contain atomic values with no repeating groups. Each field holds a single value of a defined data type.
- **2NF** - All non-key attributes are fully dependent on the entire primary key. No partial dependencies exist since all tables use single-column primary keys.
- **3NF** - No transitive dependencies exist. For example, billing amount and payment status are attributes of Billing_Record, not derived from Patient, ensuring each fact is stored in exactly one place

4.5.3 Logical Design

The following tables define the full HMS logical schema.

Key notation: PK = Primary Key, FK = Foreign Key, NN = Not Null, UQ = Unique.

Core entities

Patient

Column	Data Type	Key	Constraint	Description
--------	-----------	-----	------------	-------------

PatientId	INT	PK	NN, AUTO_INCREMENT	Unique patient identifier
PatientName	VARCHAR(255)		NN	Full legal name
PatientIdNumber	VARCHAR(13)		NN, UQ	South African ID number
Gender	VARCHAR(10)		NN	Patient gender
DateOfBirth	DATE		NN	Date of birth
PatientContact	VARCHAR(20)		NN	Mobile or email contact
Address	VARCHAR(255)		NULL	Residential address
PatientMedicalNotes	LONGTEXT		NULL	Encrypted clinical history

Table 19: Patient Entity

Doctor

Column	Data Type	Key	Constraint	Description
DoctorId	INT	PK	NN, AUTO_INCREMENT	Unique doctor identifier
FullName	VARCHAR(255)		NN	Full legal name
Specialization	VARCHAR(100)		NN	Medical specialisation
LicenseNo	VARCHAR(50)		NN, UQ	HPCSA license number
ContactNumber	VARCHAR(20)		NN	Contact number
Email	VARCHAR(255)		NN, UQ	Email address

Table 20: Doctor Entity

Nurse

Column	Data Type	Key	Constraint	Description
NurseId	INT	PK	NN, AUTO_INCREMENT	Unique nurse identifier
FullName	VARCHAR(255)		NN	Full legal name
Department	VARCHAR(100)		NN	Hospital department
LicenseNo	VARCHAR(50)		NN, UQ	SANC license number
ContactNumber	VARCHAR(20)		NN	Contact number

Table 21: Nurse Entity

Staff

Column	Data Type	Key	Constraint	Description
StaffId	INT	PK	NN, AUTO_INCREMENT	Unique staff identifier
DoctorId	INT	FK	NULL → Doctor.DoctorId	Links to Doctor if role is Doctor
NurseId	INT	FK	NULL → Nurse.NurseId	Links to Nurse if role is Nurse
Role	VARCHAR(20)		NN, CHECK (Doctor/Nurse/Admin)	Staff role type
Status	VARCHAR(20)		NN, DEFAULT Active	Active or Inactive
CreatedBy	INT		NULL	Super Admin who created record

Table 22: Staff Entity

Operational entities

Appointment

Column	Data Type	Key	Constraint	Description
ApptId	INT	PK	NN, AUTO_INCREMENT	Unique appointment identifier
PatientId	INT	FK	NN → Patient.PatientId	Linked patient
DoctorId	INT	FK	NN → Doctor.DoctorId	Attending doctor
AppointmentDate	DATETIME		NN	Scheduled date and time
Status	VARCHAR(20)		DEFAULT Scheduled	Scheduled / Completed / Cancelled
Comments	VARCHAR(255)		NULL	Patient or doctor notes

*Table 23: Appointment Entity***Prescription**

Column	Data Type	Key	Constraint	Description
RxId	INT	PK	NN, AUTO_INCREMENT	Unique prescription identifier
PatientId	INT	FK	NN → Patient.PatientId	Patient receiving prescription
DoctorId	INT	FK	NN → Doctor.DoctorId	Issuing doctor
NurseId	INT	FK	NN → Nurse.NurseId	Attending nurse
Medication	VARCHAR(255)		NN	Medicine name
Dosage	VARCHAR(100)		NN	Frequency and strength
Quantity	INT		NN, CHECK > 0	Units dispensed
IssuedDate	DATE		NN, DEFAULT CURRENT_DATE	Date of issue

*Table 24: Prescription Entity***Billing_Record**

Column	Data Type	Key	Constraint	Description
BillId	INT	PK	NN, AUTO_INCREMENT	Unique billing identifier
PatientId	INT	FK	NN → Patient.PatientId	Billed patient
Amount	DECIMAL(10,2)		NN, CHECK > 0	Total invoice amount
PaymentStatus	VARCHAR(20)		NN, DEFAULT Owing	Paid / Owing / Account
AccountBalance	DECIMAL(10,2)		NN, DEFAULT 0, CHECK >= 0	Outstanding balance
AccountTerm	INT		NULL	Payment plan months (1-12)
BillingDate	DATE		NN, DEFAULT CURRENT_DATE	Invoice date

Table 25: Billing records Entity
Inventory and supply chain
Inventory

Column	Data Type	Key	Constraint	Description
ItemId	INT	PK	NN, AUTO_INCREMENT	Unique item identifier
ItemName	VARCHAR(255)		NN	Medicine or supply name
StockLevel	INT		NN, DEFAULT 0, CHECK ≥ 0	Current quantity on hand
ReorderPoint	INT		NN, CHECK ≥ 0	Minimum threshold for alert
UnitPrice	DECIMAL(10,2)		NN, CHECK > 0	Cost per unit

Table 26: Inventory entity

Supplier

Column	Data Type	Key	Constraint	Description
SupplierId	INT	PK	NN, AUTO_INCREMENT	Unique supplier identifier
SupplierName	VARCHAR(255)		NN	Company name
ContactEmail	VARCHAR(255)		NN, UQ	Email for purchase orders
Phone	VARCHAR(20)		NN	Contact number

Table 27: Supplier entity

Purchase_Order

Column	Data Type	Key	Constraint	Description
PurchaseOrderId	INT	PK	NN, AUTO_INCREMENT	Unique PO identifier
ItemId	INT	FK	NN $\rightarrow$ Inventory.ItemId	Ordered inventory item
SupplierId	INT	FK	NN $\rightarrow$ Supplier.SupplierId	Fulfilling supplier
Quantity	INT		NN, CHECK > 0	Units ordered
TotalCost	DECIMAL(10,2)		NN, CHECK > 0	Quantity $\times$ UnitPrice
Status	VARCHAR(20)		NN, DEFAULT Pending	Pending / Confirmed / Received
OrderDate	DATE		NN, DEFAULT CURRENT_DATE	Date PO was raised

Table 28: Purchase Order Entity

Stock_Receipt

Column	Data Type	Key	Constraint	Description
ReceiptId	INT	PK	NN, AUTO_INCREMENT	Unique receipt identifier
PurchaseOrderId	INT	FK	NN $\rightarrow$ Purchase_Order.PurchaseOrderId	Linked purchase order
QuantityReceived	INT		NN, CHECK > 0	Actual units delivered
ReceivedDate	DATE		NN, DEFAULT CURRENT_DATE	Date stock was received

Table 29: Stock receipt entity

HR and shift management**Staff_Shift**

Column	Data Type	Key	Constraint	Description
ShiftId	INT	PK	NN, AUTO_INCREMENT	Unique shift identifier
StaffId	INT	FK	NN → Staff.StaffId	Assigned staff member
StartTime	DATETIME		NN	Shift start date and time
EndTime	DATETIME		NN, CHECK > StartTime	Shift end date and time
Role	VARCHAR(20)		NN	Role during shift
Status	VARCHAR(20)		NN, DEFAULT Scheduled	Scheduled / Completed / Cancelled

*Table 30: Staff shift entity***Supporting entities: Audit and Sync****Audit_Log**

Column	Data Type	Key	Constraint	Description
LogId	INT	PK	NN, AUTO_INCREMENT	Unique log entry identifier
StaffId	INT	FK	NN → Staff.StaffId	Staff member who performed action
Action	VARCHAR(255)		NN	Description of the action taken
Timestamp	TIMESTAMP		NN, DEFAULT CURRENT_TIMESTAMP	Exact time of action
IP_Address	VARCHAR(45)		NN	Network address of device used

*Table 31: Audit Log entity***SyncLog**

Column	Data Type	Key	Constraint	Description
LogId	INT	PK	NN, AUTO_INCREMENT	Unique sync entry identifier
TableName	VARCHAR(50)		NN	Table containing the offline change
RecordId	INT		NN	ID of the changed record
SyncStatus	VARCHAR(10)		NN, DEFAULT Local	Local (cached) or Synced (uploaded)
Timestamp	TIMESTAMP		NN, DEFAULT CURRENT_TIMESTAMP	Time of local capture
IP_Address	VARCHAR(45)		NN	Device IP during offline mode

*Table 32: SyncLog Entity***Relationship Mapping**

The following table summarises all foreign key relationships in the logical schema, mapping each child table back to its parent and the business rules enforced.

HMS Logical Relationship Map

Parent table	Child table	Type	FK column	Business rule enforced
Patient	Appointment	1:M	PatientId	A patient can have many appointments
Doctor	Appointment	1:M	DoctorId	A doctor attends many appointments
Patient	Prescription	1:M	PatientId	A patient can receive many prescriptions
Doctor	Prescription	1:M	DoctorId	A doctor can issue many prescriptions
Nurse	Prescription	1:M	NurseId	A nurse attends to many prescriptions
Patient	Billing_Record	1:M	PatientId	A patient can have many billing records
Doctor	Staff	1:1	DoctorId	A doctor maps to one staff record
Nurse	Staff	1:1	NurseId	A nurse maps to one staff record
Staff	Staff_Shift	1:M	StaffId	A staff member can have many shifts
Staff	Audit_Log	1:M	StaffId	A staff member generates many audit entries
Inventory	Purchase_Order	1:M	ItemId	An item can appear in many purchase orders
Supplier	Purchase_Order	1:M	SupplierId	A supplier fulfils many purchase orders
Purchase_Order	Stock_Receipt	1:M	PurchaseOrderId	A PO can result in many stock receipts

Table 33: Logical relationship map

4.6 Program Design

The pseudo code is written in a PHP-style syntax to reflect the actual backend implementation language, making it directly relatable to the development phase. Each module corresponds to a functional requirement (FR) defined in [Section 3.5](#)

FR1: User Login

```

START
INPUT username
INPUT password
IF username and password match database records THEN
    DISPLAY "Login Successful"
    GRANT system access
ELSE
    DISPLAY "Invalid Username or Password"
    DENY access
END IF
STOP

```

FR2: Register New Patient

```

START
INPUT patient details
    - Patient ID
    - Name
    - Date of Birth
    - Gender
    - Contact Number
    - Address
CHECK if Patient ID already exists
IF Patient ID exists THEN

```

```
        DISPLAY "Patient Already Registered"
ELSE
    SAVE patient information
    DISPLAY "Registration Successful"
END IF
STOP
```

FR3: Book Appointment

```
START
INPUT Patient ID
INPUT Doctor ID
INPUT Appointment Date
CHECK doctor's availability
IF doctor is available THEN
    CREATE appointment
    SAVE appointment details
    DISPLAY "Appointment Booked Successfully"
ELSE
    DISPLAY "Selected Time Slot Unavailable"
END IF
STOP
```

FR4: Update Patient Records

```
START
INPUT Patient ID
SEARCH patient record
IF record found THEN
    DISPLAY current details
    INPUT updated information
    SAVE changes
    DISPLAY "Record Updated Successfully"
ELSE
    DISPLAY "Patient Record Not Found"
END IF
STOP
```

FR5: Generate Medical Report

```
START
INPUT Patient ID
SEARCH patient records
IF records exist THEN
    GENERATE medical report
    DISPLAY report
    PRINT or SAVE report
ELSE
    DISPLAY "No Records Found"
END IF
STOP
```

Module Summary

The table below summarises all pseudo code modules to be included in code and their corresponding functional requirements.

FR / NFR	Module	Purpose
FR-01	registerPatient()	Patient registration with POPIA validation
FR-01	updatePatientProfile()	Patient profile update with approval workflow
FR-02	issuePrescription()	Digital prescription issuance with audit log
FR-03	scheduleAppointment()	Appointment creation with availability check
FR-03	requestReschedule()	Patient reschedule request and doctor approval
FR-03	approveReschedule()	Doctor approves or declines reschedule
FR-04	generateInvoice()	In-patient billing record and invoice generation
FR-04	processPayment()	Payment processing — full, partial, account plan
FR-05	checkInventoryLevels()	Scheduled stock check with early warning alerts
FR-05	createPurchaseOrder()	Auto-generate PO for low-stock items
FR-05	assignStaffShift()	Super Admin shift assignment with conflict check
FR-06	createStaffProfile()	Doctor/Nurse profile creation with license check
FR-06	removeStaffProfile()	Staff deactivation — preserves audit trail
NFR-03	logOfflineRecord()	Captures offline data entries to SyncLog
NFR-03	syncToCloud()	Reconciles local SyncLog records to Azure cloud

4.7 Interface Design

Home\Landing Page

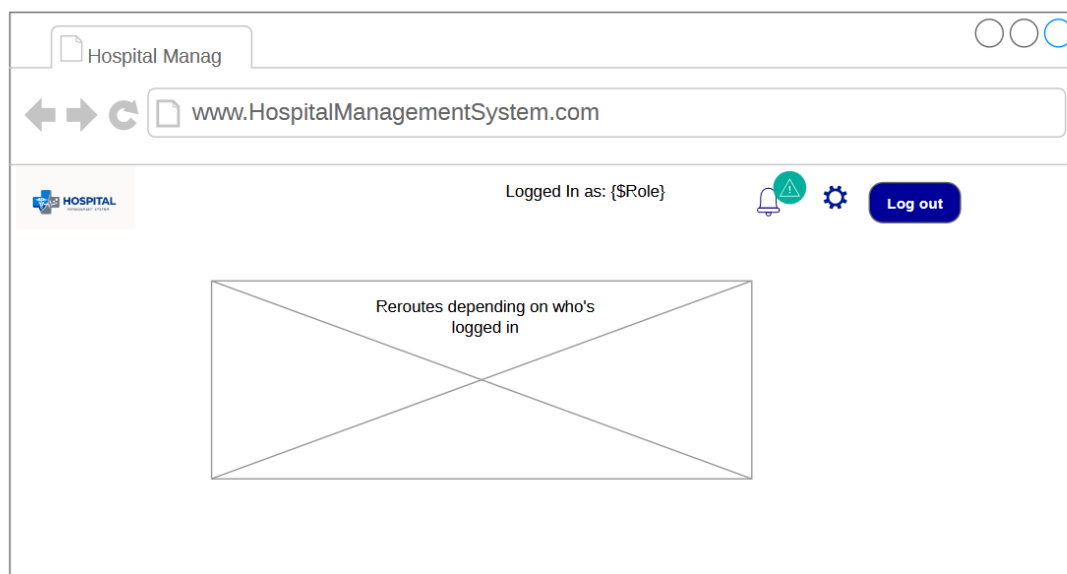


Figure 4: Landing Page UI

The screenshot shows a web browser window titled "Hospital Mana" with the address bar displaying "https://www.HospitalManagementSystem\SignIn". The page features a "HOSPITAL" logo on the left. The main content area is titled "Sign In" and contains the following elements:

- User Name:** A text input field with a red asterisk indicating it is required.
- Password:** A text input field with a red asterisk indicating it is required.
- Role:** A dropdown menu.
- SIGN IN:** A blue button.
- Forgot Password?:** A blue link.
- Don't have an account:** Text label above the SIGN UP button.
- SIGN UP:** A blue button.
- © 2026 Hospital Management System:** Footer text.

Figure 5: Login UI

The screenshot shows a web browser window titled "Hospital Mana" with the address bar displaying "https://www.HospitalManagementSystem\PatientRegistration". The page features a "HOSPITAL" logo on the left. The main content area is titled "Create Profile" and contains the following elements:

- First Name:** A text input field.
- Last Name:** A text input field.
- ID/Passport:** A text input field.
- Contact Number:** A text input field.
- Date of Birth:** A date picker showing "MM/DD/YYYY".
- Medical History:** A text area with the placeholder text "Enter brief medical history ..".
- Save:** A blue button.
- Cancel:** A red button.

Figure 6: Patient Registration UI

The screenshot shows a web browser window titled "Hospital Man" with the URL `https://www.HospitalManagementSystem/Appoinments`. The page features a header with the "HOSPITAL" logo, a "Schedule Appointments" button, and a "Logged In as: {Role}" status. There are also notification and settings icons, and a "Log out" button. The main content area is divided into two sections. On the left, under "Calendar View", there is a placeholder box. Below it, the "Appointment Request" section contains three input fields: "Patient:" (a dropdown menu), "Doctor:" (a dropdown menu), and "Date:" (a date picker). A "Book Appointment" button is positioned below these fields. On the right, there is a placeholder box labeled "Booking confirmation".

Figure 7: Schedule Appointment UI

The screenshot shows a web browser window titled "Hospital Manag" with the URL `https://www.HospitalManagementSystem/PatientPortal`. The page features a header with the "HOSPITAL" logo, a "Patient Information" button, and a "Logged In as: {Role}" status. There are also notification and settings icons, and a "Log out" button. The main content area is divided into two columns. The left column contains four sections, each with a placeholder box: "Patient Information", "Upcoming Appointments", "Medical history", and "Prescriptions". The right column contains a section titled "Invoices and Payments" with a placeholder box. Below this, there are two buttons: "Make Payment" and "Download Receipt".

Figure 8: Patient Dashboard

The screenshot shows a web browser window titled "Hospital Manag" with the URL "https://www.HospitalManagementSystem\DoctorsPortal". The dashboard features a header with the hospital logo, a "Manage Patients Prescriptions" button, and a "Logged In as: {\$Role}" status. There are also notification and settings icons, and a "Log out" button. The main content area includes a "Search Patient" section with a search bar and a "Search Results" placeholder. To the right, there are input fields for "Medication", "Dosage", and "Quantity", followed by an "Issue Prescription" button. Below the search bar is a "Medical Notes" section with a text area for "Enter diagnosis and consultation notes here..." and a "Save" button. On the bottom right, there is a "View Appointment Schedule" placeholder.

Figure 9: Doctor's Dashboard

The screenshot shows a web browser window titled "Hospital Manag" with the URL "https://www.HospitalManagementSystem\NursePortal". The dashboard features a header with the hospital logo, a "Manage Patients Intake" button, and a "Logged In as: {\$Role}" status. There are also notification and settings icons, and a "Log out" button. The main content area includes a "Nurse details and assigned shifts" placeholder. Below this, there are input fields for "Blood Pressure", "Temperature", and "Heart Rate". To the right, there is a large placeholder for "View and manage patients intake". At the bottom left, there is a "Notes" section with a text area and a "Save" button.

Figure 10: Nurse Dashboard

Hospital Mana

https://www.HospitalManagementSystem\BillingFinance

Billing and Finance Logged In as: {\$Role} [Notification] [Settings] [Log out]

Patient:

Patient	Sam Brown
Consultation	R350
Medication	R150
Total	R500

Generate Invoice

[Placeholder for Invoice]

Figure 11: Finance/Billing Dashboard

Hospital Mana

https://www.HospitalManagementSystem\InventoryControl

Inventory Control Logged In as: {\$Role} [Notification] [Settings] [Log out]

Item Details

Item ID:

Item Name:

Stock Level:

Reorder Point:

Unit Price:

Alert ID:

Alert Date:

Save Item

Inventory Records Table

[Placeholder for Inventory Records Table]

Figure 12: Inventory Management Dashboard

The screenshot shows a web browser window titled "Hospital Mana". The address bar displays "https://www.HospitalManagementSystem\SupplierManagement". The page header includes a "Supplier Management" button, a "Logged In as: {Role}" indicator, a notification bell, a settings gear, and a "Log out" button. The main content area is titled "Supplier Details" and contains three input fields: "Supplier Name:", "Contact Email:", and "Phone:". Below these fields is a "Save Supplier" button. At the bottom, there is a placeholder for a "Supplier Records Table" represented by a large rectangle with a diagonal cross.

Figure 13: Suppliers Dashboard

The screenshot shows a web browser window titled "Hospital Mana". The address bar displays "https://www.HospitalManagementSystem\PurchaseOrder". The page header includes a "Purchase Order" button, a "Logged In as: {Role}" indicator, a notification bell, a settings gear, and a "Log out" button. The main content area contains several input fields: "Purchase Order ID:" with a value of "(Auto Generated)", "Item ID:" with a "Select Item" dropdown, "Supplier ID:" with a "Select Supplier" dropdown, "Quantity:", "Unit Price:", "Total Cost:", "Order Date:" with a calendar icon, and "Status:". Below these fields is a "Create Purchase Order" button. To the right of the input fields is a large placeholder box with a diagonal cross.

Figure 14: Inventory PO

The screenshot shows a web browser window titled "Hospital Mana" with the URL `https://www.HospitalManagementSystem\StockReceiptManagement`. The page header includes the "HOSPITAL" logo, a "Stock Receipt Management" title bar, a "Logged In as: {Role}" status, a notification bell icon, a settings gear icon, and a "Log out" button. The main content area contains a form with the following fields: "Receipt ID:" with a value of "(Auto Generated)", "Purchase Order" with a dropdown menu showing "Select PO", "Received Quantity:" with an empty input field, and "Received Date:" with an empty date picker. A "Save" button is located below the form. To the right of the form is a large square placeholder with a diagonal 'X'.

Figure 15: Inventory Stock

The screenshot shows a web browser window titled "Hospital Mana" with the URL `https://www.HospitalManagementSystem\AuditLogs`. The page header includes the "HOSPITAL" logo, an "Audit Logs" title bar, a "Logged In as: {Role}" status, a notification bell icon, a settings gear icon, and a "Log out" button. The main content area is titled "Audit Logs" and contains two rectangular placeholders with diagonal 'X's, labeled "Audit Logs" and "Sync Logs". At the bottom of the page is a blue "Export Report" button.

Figure 16: Audit Logs Dashboard

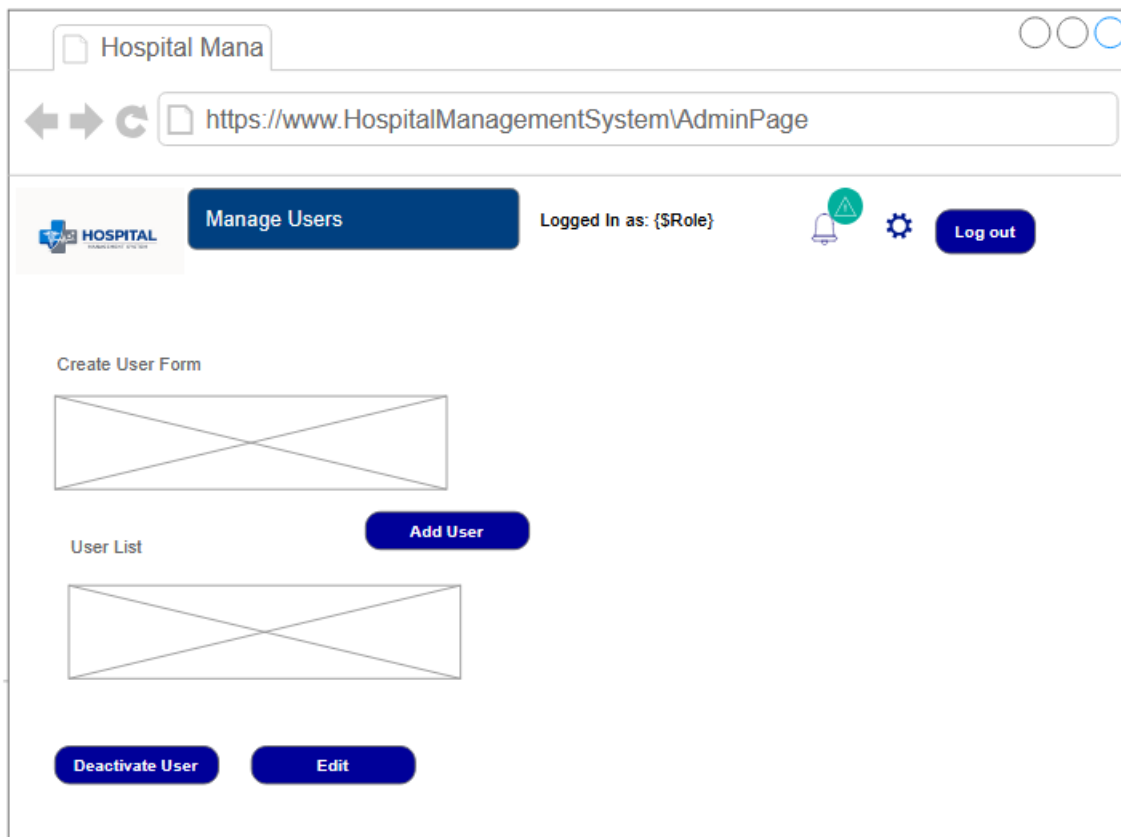


Figure 17: Admin Dashboard

4.8 Security and Back-up Design

Given that the system handles highly sensitive patient data, financial records, and staff information, security is not an optional layer, it is built into every tier of the architecture. All design decisions in must be aligned to the non-functional requirements defined in [Section 3.6 \(NFR-1 through NFR-6\)](#) and the South African regulatory frameworks governing healthcare data, nurses and doctors council.

4.8.1 Software Security

a) Authentication and Access Control

The HMS enforces a multi-layered authentication model to ensure only authorised users can access the system and its data.

b) Role-Based Access Control (RBAC)

Every HMS function checks the authenticated user's role before executing. The following access matrix defines what each role can do:

Role	Patient data	Prescriptions	Billing	Inventory	Staff profiles	Audit logs
Super Admin	Full CRUD	Read only	Full CRUD	Full CRUD	Full CRUD	Read only
Admin	Read / Update	Read only	Full CRUD	Read / Update	Read only	Read only
Doctor	Read / Update	Create / Read	Read only	Read only	Read only	Own actions
Nurse	Read / Update	Read only	Read only	Read / Update	Read only	Own actions
Patient	Own record	Own records	Own records	None	None	None
Pharmacist	Read only	Dispense only	None	Update stock	None	None

Table 34: RBAC matrix

c) Data Encryption

The HMS will apply two encryption levels: data in transit and data at rest, to protect sensitive patient and financial information.

Data in transit

- **HTTPS / TLS 1.2/1.3:** All communication between client browsers and the web server is encrypted via TLS 1.2/1.3 on port 443. Unencrypted HTTP requests are automatically redirected to HTTPS.
- **TLS sync channel:** All data synchronization between the on-premise MySQL server and the Azure cloud replica is transmitted over a dedicated TLS-secured tunnel, preventing interception during cloud sync.
- **API gateway encryption:** All external API calls (payment gateway, health systems, audit service) are made over SSL/TLS with certificate validation enforced.

Data at rest

- **PatientMedicalNotes:** The PatientMedicalNotes field (LONGTEXT) stores encrypted clinical data. AES-256 encryption is applied at the application layer before writing to the database, ensuring medical records are unreadable even with direct database access.
- **Password hashing:** All staff passwords are hashed using bcrypt with a salt factor of 12 before storage. Plain text passwords are never stored.
- **Azure Blob storage:** Audit log archives stored in Azure Blob Storage are encrypted at rest using Azure Storage Service Encryption (SSE) with AES-256.

d) Input Validation and SQL Injection Prevention

All user input is validated and sanitised before being processed or stored. The HMS uses prepared statements with parameterised queries throughout the PHP backend, completely eliminating the risk of SQL injection attacks.

e) Session Management

Session security ensures that authenticated sessions cannot be hijacked or replayed by unauthorized parties.

- **Session timeout:** All HMS sessions automatically expire after 15 minutes of inactivity, requiring the user to re-authenticate.
- **Session token regeneration:** A new session token is generated on every successful login to prevent session fixation attacks.
- **Secure cookie flags:** Session cookies are set with HTTP Only and Secure flags, preventing JavaScript access and ensuring transmission over HTTPS only.
- **Concurrent session prevention:** A user cannot be logged in from more than one device simultaneously. A new login from a second device invalidates the existing session.

f) Audit Logging and Compliance

The HMS maintains a comprehensive audit trail of all system activity through the Audit_Log table. This satisfies NFR-6 (Auditability) and supports POPIA compliance by providing a verifiable record of who accessed or modified patient data, when, and from which device. according to (The Medical Protection Society Limited, 2025) and (Marais, 2018), medical records can be stored for up to 6 and that is rule applied for the HMS.

Event type	Logged automatically	Fields captured	Retention
User login / logout	Yes - on every auth event	StaffId, Action, Timestamp, IP	6 years
Patient record access	Yes - on every read/write	StaffId, PatientId, Action, Timestamp	6 years
Prescription issuance	Yes - trigger on INSERT	DoctorId, RxId, Timestamp, IP	6 years
Appointment created/updated	Yes - trigger on INSERT	StaffId, ApptId, Timestamp, IP	6 years
Billing record modified	Yes - on payment processing	StaffId, BillId, Action, Timestamp	6 years
Staff profile created/removed	Yes - Super Admin actions	SuperAdminId, StaffId, Action	Permanent
Failed login attempts	Yes - on auth failure	Username, IP, Timestamp	2 years
SyncLog reconciliation	Yes - on cloud sync	LogId, SyncStatus, Timestamp, IP	3 years

Table 35: Audit Log Compliance matrix

g) Regulatory Compliance Summary

The following table maps each South African regulatory requirement to the HMS security control that satisfies it:

Regulation	Requirement	HMS control
POPIA — Section 19 (GCIS, 2013)	Secure processing of personal information	AES-256 encryption, TLS 1.2/1.3, RBAC
POPIA — Section 22	Notify data subjects of security compromise	Audit log alerting, incident notification module

Regulation	Requirement	HMS control
(GCIS, 2013)		
POPIA — Section 14 (GCIS, 2013)	Restrict access to authorised parties only	Role-based access control, 2FA
NHA — Section 13 (RSA Government, 2026)	Patient confidentiality must be maintained	Encrypted PatientMedicalNotes, access logging
HPCSA Guidelines (Health Professions Council of South Africa, 2026)	Clinical records must be accurate and auditable	Immutable audit trail, no-delete policy on records
HPCSA Guidelines (Health Professions Council of South Africa, 2026)	Prescription records must be traceable	Prescription audit trigger, RxId audit log entry

Table 36: Regulatory Compliance

4.8.2 Backup and Recovery Design

a) Backup Strategy

The HMS backup strategy follows a three-tier approach; on-premise daily snapshots, Azure cloud replication, and offline-first Sync Log continuity. This is to ensure that data is never lost regardless of the failure scenario.

Backup tier	Method	Frequency	Retention	Recovery point
On-premise snapshot	Full + incremental MySQL dump	Nightly (00:00)	30 days	24 hours max
Azure cloud replication	Real-time TLS database sync	Near real-time	90 days	Minutes
Azure Blob archive	Encrypted audit log archival	Daily (automated)	7 years	On request
SyncLog offline cache	Service Worker / IndexedDB	On every transaction	Until synced	Zero loss

Table 37: Back-up Strategy

b) Disaster Recovery Procedure

The HMS disaster recovery procedure defines the steps to restore full system operation following a critical failure. Three failure scenarios are considered:

Scenario 1: Database server failure

Recovery steps — estimated RTO: 2 hours

- STEP 1: Detect failure via automated health monitor alert
- STEP 2: Switch application server to point to Backup Server (Standby)
- DB: switchConnection('backup_server')

- STEP 3: Verify Backup Server data integrity via checksum
- STEP 4: Notify Super Admin and IT team of failover
- STEP 5: Restore primary server from latest nightly snapshot
- STEP 6: Re-sync primary from Backup Server to recover any gap
- STEP 7: Switch application back to primary server
- STEP 8: Log incident to Audit_Log and notify stakeholders

Scenario 2: Total on-premise failure (fire, flood, theft)

Recovery steps — estimated RTO: 4-8 hours

- STEP 1: Declare disaster recovery event - notify hospital management
- STEP 2: Provision new cloud-hosted server on Azure
- STEP 3: Restore full database from Azure SQL cloud replica
- AzureDB: restoreSnapshot('latest')
- STEP 4: Update DNS to point HMS domain to Azure-hosted server
- STEP 5: Verify all data integrity - run checksum against last known backup
- STEP 6: Re-enable staff access - force password resets for all users
- STEP 7: Notify all staff of temporary cloud-hosted access URL
- STEP 8: Log disaster recovery event to Audit_Log

Scenario 3: Extended internet outage

Recovery steps — no data loss expected

- STEP 1: System automatically switches to offline mode
- ServiceWorker:activateOfflineMode()
- STEP 2: All transactions continue normally - data cached via IndexedDB
- STEP 3: SyncLog records all changes with SyncStatus = 'Local'
- STEP 4: When connectivity restores, SyncLog manager auto-reconciles
- SyncManager:syncToCloud() // see Section 4.6 NFR-03 pseudo code
- STEP 5: Verify all SyncLog entries updated to SyncStatus = 'Synced'
- STEP 6: Log connectivity event and sync summary to Audit_Log

c) Backup Schedule and Responsibilities

The following schedule defines the automated backup operations and the responsible parties for each:

Day / Frequency	Backup type	Automated	Responsible	Verification
Sunday 00:00	Full MySQL backup	Yes	System (cron job)	Checksum auto-verified
Mon–Sat 00:00	Incremental MySQL backup	Yes	System (cron job)	Checksum auto-verified
Every transaction	SyncLog offline cache	Yes	SyncLog manager	SyncStatus check
Near real-time	Azure cloud replication	Yes	Azure sync service	Azure health monitor
Daily 02:00	Audit log archive to Blob	Yes	System (cron job)	Azure confirmation

Day / Frequency	Backup type	Automated	Responsible	Verification
Monthly	Full recovery test	No	Super Admin / IT	Manual RTO verification
Annually	Disaster recovery drill	No	Hospital IT team	Full scenario walkthrough

Table 38: Back-up schedule

5 Implementation Phase

5.1 Introduction

This section deals with the HMS system development and testing for the launch of the MVP (Minimum Viable Product) product for pilot deployment. The implementation phase takes approved system design and converts it into a fully functional software application. During this phase, the following activities occur

- developers write the source code, integrate the various system modules,
- conduct different levels of testing, and prepare the application for deployment
- user testing and business sign-off is concluded
- an issue tracker is created to track bugs and MVP changes

The objective of this phase is to ensure that the Hospital Management System (HMS) meets all functional and non-functional requirements identified during the analysis and design phases.

5.2 Coding

The following tools and technologies were used during the development/coding implementation of the HMS:

Component	Technology / Tool	Purpose
Frontend	HTML, CSS, JavaScript	User interface and client-side interactions
Backend	PHP	Business logic, FR module processing, API routing
Database	MySQL	Relational data storage, 3NF compliant
Local server	XAMPP (Apache + MySQL)	Local development and testing environment
Code editor	Visual Studio Code (VS Code)	Primary development IDE
Version control	Git / GitHub	Source code management and team collaboration
AI assistance	IDE GitHub Copilot & Claude Code	Code completions and development support
Cloud platform	Microsoft Azure (Student)	Cloud deployment and Azure SQL sync target

Table 39: Development Environment

The following key decisions were made during the implementation phase:

- **PDO prepared statements:** All database queries use PHP Data Objects (PDO) with parameterised queries throughout the backend, eliminating SQL injection vulnerabilities
- **Session-based authentication:** PHP sessions manage user authentication state, with role stored in the session for RBAC enforcement on every protected page.
- **Modularized structure:** The codebase is organized into well-structured files to manage rules, scripts, config, pages, and styling folders, keeping the code maintainable.
- **CSRF protection:** All HTML forms include a hidden CSRF token, validated on the server-side before any data is processed or written to the database (PortSwigger, 2026) (KirstenS, 2026).

- **Offline-first:** Service Worker and IndexedDB (Silva, 2025) are implemented at the client layer to enable offline data capture, with the SyncLog PHP module handling reconciliation to Azure on reconnection.

5.3 Testing

Testing Objectives

The main objectives of system testing are:

- To verify that all HMS modules (Login, Dashboard, Patient Management) work together correctly
- To ensure the system meets all functional requirements
- To check data integrity between the application and MySQL database
- To identify errors, bugs, or unexpected system behaviour
- To confirm security measures (e.g., authentication, SQL injection protection)

The system testing results indicate that the HMS performs effectively under all tested conditions.

Security Evaluation

- The system uses prepared statements, which protects against SQL injection attacks
- Only authenticated users can access the dashboard
- Invalid login attempts are blocked with error messages
- No direct database access is exposed to the user interface
- All forms use CSRF tokens to authenticate
- Every data mutation writes a row to the audit log for tracking changes
- Sessions are used strictly and regenerate ID on login.
- Sync operations are authenticated with a per-session token and verified again for synching

Performance Evaluation

- System responds quickly during login and patient data operations
- Database queries execute efficiently
- No system crashes observed during testing

Usability Evaluation

- Interface is design to be simple and user-friendly with a right-sided menu pane
- Buttons are clearly labelled supported with the use simple Unicode icons using (Vertex42, 2024) codes.
- All required fields are clearly marked with validations triggered on both client-side and server-side for correctness
- Users can navigate easily between modules

Conclusion

The system testing confirms that the Hospital Management System, as an MVP release is:

- Functional

- Stable
- Secure
- User-friendly
- Ready for deployment or further enhancement

All test cases were successfully executed and issues identified as bugs will be resolved immediately through an issue tracker.

Any other feature to enhance the system will be done in future post MVP/Pilot release, and these include but no limited to;

- Two factor authentication via 3rd Party service like Google Authenticator
- Enhancing 3rd party API for payments and other services
- outgoing emails for user accounts and changes as a security measure on profile changes, password requests. log in and other services provided by the HMS
- AES-256 encryption on medical notes is deferred, encryption not yet applied

5.4 System Testing (Test Case, Evaluation of the testing results)

Below are the test case plan for full functional system testing

Ref #	Req. Reference	Scenario	Title / Description	Expected Result	Pass / Fail	Comment	Executed By
FR-01 — Patient Data Management							
TCR-0001	FR-01	Straight through path	Register new patient with valid SA ID	Patient record saved. Unique PatientId generated. Confirmation notification sent to patient.	Pass		
TCR-0002	FR-01	Validation	Register patient with invalid SA ID (less than 13 digits)	System rejects submission. Error: "Invalid South African ID number" displayed.	Pass		
TCR-0003	FR-01	Duplicate	Register patient with ID number already in the system	System detects duplicate. Error: "Patient already registered". No new record created.	Pass		
TCR-0004	FR-01	Auth check	Attempt patient registration while logged in as Doctor role	System blocks action. Error: "Unauthorised access" displayed. No record created.	Pass		
TCR-0005	FR-01	Empty fields	Submit registration form with required fields left empty	System rejects submission. Error: "Required fields are missing" displayed.	Pass		
TCR-0006	FR-01	Update request	Patient submits a profile update request	Update request saved with status Pending. Admin/Nurse receives notification for approval.	Pass		
TCR-0007	FR-01	Approve update	Admin approves a pending patient profile update request	Patient profile updated in database. Patient receives confirmation notification.	Pass		
FR-02 — Medicine / Prescription Issuance							
TCR-0008	FR-02	Straight through path	Doctor issues a valid prescription with all required fields	Prescription saved with unique RxId. Audit log entry created. Pharmacist notified for dispensing.	Pass		

Ref #	Req. Reference	Scenario	Title / Description	Expected Result	Pass / Fail	Comment	Executed By
TCR-0009	FR-02	Auth check	Nurse attempts to issue a prescription	System blocks action. Error: "Only Doctors can issue prescriptions" displayed.	Pass		
TCR-0010	FR-02	Invalid qty	Doctor submits prescription with quantity set to zero	System rejects submission. Error: "Invalid prescription details" displayed.	Pass		
TCR-0011	FR-02	Invalid ref	Doctor issues prescription with an invalid PatientId	System rejects submission. Error: "Patient not found". No record created.	Pass		
TCR-0012	FR-02	Audit trail	Verify audit log entry created after prescription issuance	Audit_Log contains entry with DoctorId, RxId, action, timestamp, and IP address.	Pass		
FR-03 — Appointment Scheduling							
TCR-0013	FR-03	Straight through path	Patient books appointment with an available doctor	Appointment saved with status Scheduled. Patient and Doctor both receive confirmation.	Pass		
TCR-0014	FR-03	Conflict check	Book appointment for doctor already booked at same date/time	System detects conflict. Error: "Doctor not available at requested time" displayed.	Pass		
TCR-0015	FR-03	Auth check	Pharmacist attempts to schedule an appointment	System blocks action. Error: "Unauthorised access" displayed.	Pass	Access not granted	
TCR-0016	FR-03	Reschedule req	Patient requests appointment reschedule to a new date	Reschedule request submitted. Doctor receives notification. Original appointment unchanged.	Pass		
TCR-0017	FR-03	Approve resch.	Doctor approves a patient reschedule request	Appointment date updated to new date. Patient notified. Audit log entry created.	Pass		
TCR-0018	FR-03	Decline resch.	Doctor declines a patient reschedule request	Original appointment date remains unchanged. Patient notified of decline.	Pass		
FR-04 — Billing Management							

Ref #	Req. Reference	Scenario	Title / Description	Expected Result	Pass / Fail	Comment	Executed By
TCR-0019	FR-04	Generate invoice	Admin generates invoice on patient discharge	Billing record created with status Owing. Itemised invoice sent to patient. BillId generated.	Pass		
TCR-0020	FR-04	Full payment	Patient makes full payment on outstanding invoice	PaymentStatus updated to Paid. AccountBalance set to zero. Payment confirmation sent.	Pass		
TCR-0021	FR-04	Partial payment	Patient makes partial payment on outstanding invoice	AccountBalance reduced by amount paid. PaymentStatus remains Owing. Updated balance displayed.	Pass		
TCR-0022	FR-04	Account plan	Patient selects 12-month account payment plan	PaymentStatus set to Account. AccountTerm = 12. Monthly instalment calculated and sent to patient.	Pass		
TCR-0023	FR-04	Auto paid	Patient final instalment reduces AccountBalance to zero	Database trigger fires automatically. PaymentStatus updated to Paid. AccountBalance = 0.	Pass		
TCR-0024	FR-04	Invalid amount	Admin generates invoice with amount of zero	System rejects submission. Error: "Amount must be a positive number" displayed.	Pass		
FR-05 — Risk Management Tracking							
TCR-0025	FR-05	Stock alert	Inventory item stock level drops to or below reorder point	Early warning alert sent to Admin. Audit log entry created with ItemId and timestamp.	Pending	Passed unit test, dummy data used	
TCR-0026	FR-05	Stock OK	Inventory item stock level is above reorder point	No alert generated. Scheduled check completes with no action required.	Pass		
TCR-0027	FR-05	Create PO	Admin creates purchase order for a low-stock inventory item	Purchase Order saved with status Pending. Digital PO sent to supplier email. Total cost calculated.	Pass		

Ref #	Req. Reference	Scenario	Title / Description	Expected Result	Pass / Fail	Comment	Executed By
TCR-0028	FR-05	Receive stock	Admin confirms stock receipt against an existing purchase order	Inventory StockLevel updated. Purchase Order status set to Received. Trigger fires automatically.	Pass		
TCR-0029	FR-05	Assign shift	Super Admin assigns shift to an available staff member	Shift saved with status Scheduled. Staff member receives shift notification.	Pass		
TCR-0030	FR-05	Shift conflict	Super Admin assigns shift overlapping existing shift for same staff	System detects conflict. Error: "Shift conflict detected" displayed. No shift created.	Pass		
TCR-0031	FR-05	Understaffing	Shift assignment results in staff count below minimum required	Shift saved but Super Admin receives understaffing warning notification.	Pending	Passed unit test, dummy data used	
FR-06 — Doctor / Nurse Management							
TCR-0032	FR-06	Create doctor	Super Admin creates new Doctor profile with valid HPCSA license	Doctor record saved. Staff record created with role Doctor, status Active. Audit log entry created.	Pass		
TCR-0033	FR-06	Create nurse	Super Admin creates new Nurse profile with valid license number	Nurse record saved. Staff record created with role Nurse, status Active. Audit log entry created.	Pass		
TCR-0034	FR-06	Dup. license	Super Admin registers a license number already in the system	System rejects submission. Error: "License number already registered" displayed.	Pass		
TCR-0035	FR-06	Auth check	Admin (not Super Admin) attempts to create a staff profile	System blocks action. Error: "Only Super Admin can create staff profiles" displayed.	Pass		
TCR-0036	FR-06	Deactivate	Super Admin deactivates an existing staff profile	Staff status set to Inactive. All linked records preserved. Audit log entry created.	Pass		
TCR-0037	FR-06	Preserve records	Verify clinical notes remain intact after staff deactivation	PatientMedicalNotes and Audit_Log entries linked to deactivated staff remain in database unchanged.	Pass		

Ref #	Req. Reference	Scenario	Title / Description	Expected Result	Pass / Fail	Comment	Executed By
NFR-01 — Security							
TCR-0038	NFR-01	2FA login	User logs in with correct password and valid OTP code	Session created. User directed to role-appropriate dashboard.	Not Ready	To be done in phase 2 of the project after MVP release	
TCR-0039	NFR-01	Wrong OTP	User enters correct password but incorrect OTP code	Login rejected. Error: "Invalid or expired OTP" displayed. Session not created.	Not Ready	To be done in phase 2 of the project after MVP release	
TCR-0040	NFR-01	Account lock	User enters wrong password 5 consecutive times	Account locked after 5th failed attempt. User cannot log in until unlocked by Admin.	Not Ready	To be done in phase 2 of the project after MVP release	
TCR-0041	NFR-01	Session timeout	User leaves HMS idle for more than 15 minutes	Session destroyed automatically. User redirected to login. Message: "Session expired".	Pass		
TCR-0042	NFR-01	RBAC check	Patient attempts to access billing management screen	Access denied. User redirected to patient dashboard. Error: "Unauthorised access".	Pass		
TCR-0043	NFR-01	SQL injection	Attacker enters SQL code into the patient search field	Prepared statement neutralises the attack. No data returned. No database error exposed.	Pass		
TCR-0044	NFR-01	CSRF token	Form submitted without a valid CSRF token	Request blocked. Error: "Invalid CSRF token" displayed. No data written to database.	Pass		
TCR-0045	NFR-01	Encryption	Verify PatientMedicalNotes stored encrypted in database	Direct DB query shows encrypted ciphertext. Decryption only accessible via authorised app layer.	Pass		
NFR-03 — Offline-First Sync							

Ref #	Req. Reference	Scenario	Title / Description	Expected Result	Pass / Fail	Comment	Executed By
TCR-0046	NFR-03	Offline entry	Staff creates patient record while system is offline	Record saved locally via IndexedDB. SyncLog entry created with SyncStatus = Local.	In Progress	Local testing passed, pending deployment tests	
TCR-0047	NFR-03	Sync on reconnect	Network restored after offline data entry	SyncLog manager auto-triggers. All Local records pushed to Azure SQL. SyncStatus updated to Synced.	In Progress	Local testing passed, pending deployment tests	
TCR-0048	NFR-03	Sync failure	Azure cloud unavailable during sync attempt	Sync failure logged to error log. SyncStatus remains Local. Retry on next reconnection cycle.	In Progress	Local testing passed, pending deployment tests	
TCR-0049	NFR-03	Data integrity	Verify synced records match original offline entries	All field values in Azure SQL replica match locally captured records exactly. Zero data loss confirmed.	In Progress	Local testing passed, pending deployment tests	

Table 40: HMS Test Cases

5.5 Installation

This section provides a complete step-by-step installation guide for setting up and running the Hospital Management System (HMS) on a local machine using XAMPP. The guide covers all prerequisites, database setup, configuration, and verification steps required to have the system fully operational.

5.5.1 System Requirements

The following minimum system requirements must be met before installation:

Component	Minimum requirement	Recommended
Operating System	macOS 11 (Big Sur) or Windows 10	macOS 13+ or Windows 11
Browser	Google Chrome / Firefox (latest)	Google Chrome
XAMPP version	8.0 or higher	8.2 (PHP 8.2, MySQL 8.0)
PHP version	PHP 8.0 or higher	PHP 8.2
MySQL version	MySQL 5.7 or higher	MySQL 8.0

Table 41: System Requirements

5.5.2 Installation Steps

Attached to the proposal is the following:

- Full program ZIP file (embedded) – See [Appendices B](#)

Step 1: Install and Launch XAMPP

Once installed, open the XAMPP Control Panel, and start both the Apache and MySQL services

Step 2: Clone the HMS Repository from GitHub/Shared HMS.ZIP

Unzip file and copy the folder 'HMS' inside htdocs containing all project files.

Step 3: Configure the Database Connection

Copy the secrets template and fill in your MySQL credentials:

Open the file “/HMS/config/external_config.php” and update the following credentials to match your local XAMPP setup:

```
<?php
define('DB_HOST', 'localhost');
define('DB_NAME', 'hms_db');           // database name
define('DB_USER', 'root');             // default XAMPP username
define('DB_PASS', '');                 // default XAMPP password (blank)
?>
```

Step 4: Run the set up page

Open your browser and visit: <http://localhost/HMS/setup.php>. This will:

- Create the hms_db database
- Run the full schema (17 tables, 5 triggers)
- Seed all demo data (users, patients, appointments, prescriptions, billing, inventory, suppliers, shifts)

Upon completion, you will see a green success log ending with '==== Setup complete! ===='.

Important: After setup, delete or block access to 'setup.php, it re-seeds the database on every run.

Step 5: Log in

Open <http://localhost/HMS/>

Note: this could be running on a configured port in XAMP, e.g. <http://localhost:2323/HMS/>

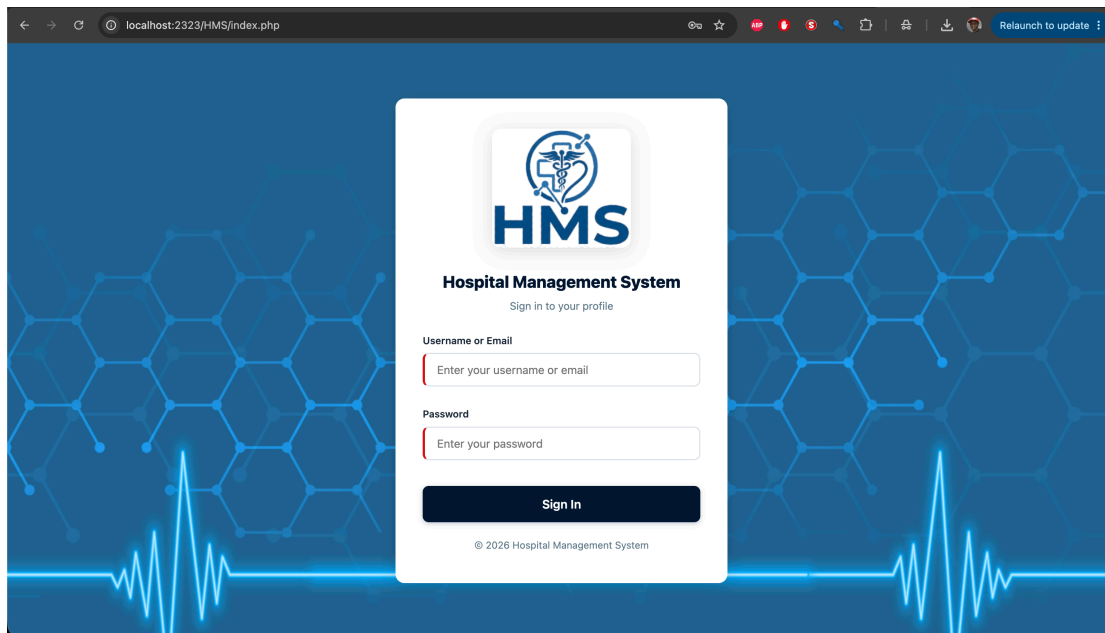


Figure 18: HMS Log in Page

Testing Data – Seeded at Step 4

The following details can be used to test the site

#	Username	Password	Role
1	superadmin	d3f@uL+Ed1	SuperAdmin
2	admin1	d3f@uL+Ed1	Admin
3	DR_Kazenga	d3f@uL+Ed1	Doctor
4	DR_Lupa	d3f@uL+Ed1	Doctor
5	nurse_tim	d3f@uL+Ed1	Nurse
6	nurse_joy	d3f@uL+Ed1	Nurse
7	patient_1	d3f@uL+Ed1	Patient
8	patient_2	d3f@uL+Ed1	Patient

Table 42: Sample Profiles Test data

6 References

- The Medical Protection Society Limited, 2025. *Factsheets*. [Online]
Available at: <https://www.medicalprotection.org/southafrica/casebook-and-resources/factsheets/factsheets/sa-retention-of-medical-records>
[Accessed 5 June 2025].
- Marais, M., 2018. *HPCSA Guidelines on record-keeping*. [Online]
Available at: <https://www.profnetmedical.co.za/news/2018/hpcsa-guidelines-on-record-keeping/>
[Accessed 5 June 2025].
- GCIS, 2013. Protection of Personal Information Act, 2013. *Government Gazette of The Republic of South Africa*, 26 November, Vol. 581(No. 37067).
- RSA Government, 2026. *National Health Act 61 of 2003*. [Online]
Available at: <https://www.gov.za/documents/acts/national-health-act-61-2003-23-jul-2004>
[Accessed 5 June 2026].
- Health Professions Council of South Africa, 2026. *PAIA & POPIA MANUALS*. [Online]
Available at: <https://www.hpcsa.co.za/page/paia-popia>
[Accessed 5 June 2026].
- PortSwigger, 2026. *Cross-site request forgery (CSRF)*. [Online]
Available at: <https://portswigger.net/web-security/csrf>
[Accessed 13 June 2026].
- KirstenS, 2026. *Cross Site Request Forgery (CSRF)*. [Online]
Available at: <https://owasp.org/www-community/attacks/csrf>
[Accessed 28 June 2026].
- Silva, S., 2025. *IndexedDB: A Comprehensive Overview for Frontend Developers*. [Online]
Available at: <https://medium.com/@shashika.silva88/indexeddb-a-comprehensive-overview-for-frontend-developers-6b47a9f32e23>
[Accessed 29 June 2026].
- Vertex42, 2024. *Huge List of Unicode Symbols*. [Online]
Available at: <https://www.vertex42.com/ExcelTips/unicode-symbols.html>
[Accessed 29 June 2026].

7 Glossary

Term	Definition
2FA (Two-Factor Authentication)	A security mechanism requiring users to verify their identity using two separate factors: their password and a one-time OTP (One-Time Password) sent to their registered contact number.
Account Lockout	An HMS security control that automatically locks a user account after five consecutive failed login attempts, preventing brute-force password attacks.
Account Plan	A payment arrangement in the HMS where a patient pays their hospital bill in monthly instalments over a period of up to 12 months at no interest, supported by a government subsidy model.
AccountBalance	The outstanding amount remaining on a patient's billing record in the HMS. Automatically updated when payments are received and set to zero when the account is fully settled.
AES-256 Encryption	Advanced Encryption Standard with a 256-bit key, the encryption algorithm used by the HMS to protect PatientMedicalNotes stored in the database at rest.
Agile Methodology	A software development approach that emphasizes iterative delivery with continuous feedback and adaptability to changing requirements. The HMS project adopts Agile via the SCRUM framework.
Apache	The web server software used in the XAMPP local development environment to serve the HMS PHP application. In production, the application is hosted on Azure App Service.
Appointment	A scheduled consultation between a Patient and a Doctor. The HMS manages appointment creation, rescheduling requests, and cancellations through FR-03.
Audit Trail	A chronological record of all system activities, capturing who performed an action, what was done, when it occurred, and from which device. The HMS Audit_Log table maintains this trail for POPIA and HPCSA compliance.
Azure App Service	A Microsoft cloud platform used to host the HMS PHP web application in the cloud. Part of the Azure for Students subscription used by the project team.
Azure SQL Database	The cloud-hosted MySQL-compatible database on Microsoft Azure that serves as the HMS cloud replica. Local on-premise data is synchronized to Azure SQL via the SyncLog manager.
Bcrypt	A password hashing algorithm used in the HMS to securely store user passwords. Bcrypt applies a salt and a configurable cost factor, making brute-force attacks computationally expensive.

Billing Record	A financial record in the HMS (Billing_Record table) that captures a patient's total charges, payment status, account balance, and payment plan details for a specific hospital visit.
Clinical Notes	Written observations, diagnoses, and treatment records captured by a Doctor or Nurse during a patient consultation. Stored as encrypted PatientMedicalNotes in the HMS database.
Context Diagram	A high-level diagram showing the HMS system as a single process and its interactions with all external entities such as Patients, Doctors, Regulatory Bodies, and the Bank.
CSRF Token (Cross-Site Request Forgery Token)	A unique secret value embedded in every HMS HTML form. The server validates the token on submission, requests without a valid token are rejected, preventing cross-site forgery attacks.
Data Encryption	The process of converting data into an unreadable format to protect it from unauthorised access. The HMS encrypts PatientMedicalNotes using AES-256 and all data in transit using TLS 1.3.
Doctor	A licensed medical professional registered in the HMS with a valid HPCSA license number. Doctors issue prescriptions, attend appointments, and access patient medical histories.
Entity Relationship Diagram (ERD)	A visual representation of the HMS database structure showing all entities (tables), their attributes (columns), and the relationships between them. The HMS ERD is defined in Section 3.7.
Functional Requirement (FR)	A specific feature or behaviour the HMS must provide. The HMS has six core FRs: FR-01 Patient Data Management, FR-02 Prescription Issuance, FR-03 Appointment Scheduling, FR-04 Billing Management, FR-05 Risk Management, and FR-06 Staff Management.
Go-Live	The point at which changes on HMS system are deployed to production environment for live use in the hospital environment.
Government Subsidy	A financial support mechanism referenced in the HMS billing design that enables patients to access an interest-free 12-month payment plan for settling their hospital accounts.
Hospital Management System (HMS)	The web-based platform being developed in this project to digitalise patient records, streamline hospital workflows, and centralise hospital operations including appointments, billing, prescriptions, and inventory.
HPCSA (Health Professions Council of South Africa)	The South African statutory body that regulates health professionals. The HMS validates Doctor license numbers against HPCSA standards during staff profile creation (FR-06).
Hybrid Cloud-Edge Model	The HMS network architecture that combines on-premise local processing (hospital LAN and servers) with cloud hosting (Azure) to support both online and offline-first operation.
IndexedDB	A browser-based local database used by the HMS client layer to store data offline when internet connectivity is unavailable. Captured records are queued for synchronisation via the SyncLog.

MySQL	The relational database management system used by the HMS to store all operational data. The schema is designed to Third Normal Form (3NF) across 14 tables.
NHA (National Health Act)	South African legislation (Act 61 of 2003) governing healthcare standards, patient rights, and the management of health establishments. HMS data practices are designed to comply with the NHA.
Non-Functional Requirement (NFR)	A quality attribute the HMS must satisfy beyond its core features, such as security (NFR-1), performance (NFR-2), reliability (NFR-3), interoperability (NFR-4), scalability (NFR-5), and auditability (NFR-6).
Nurse	A licensed healthcare professional registered in the HMS with a valid SANC license number. Nurses register patients, capture in-patient notes, attend to prescriptions, and manage shift schedules.
Offline-First	An architectural design principle where the HMS continues to function fully during internet outages by caching data locally. Captured data is synchronised to the cloud once connectivity is restored.
Patient	An individual receiving healthcare services at the hospital. Registered in the HMS with a unique PatientId and 13-digit SA ID number. Patients can access their own records, appointments, and billing through the patient portal.
Payment Gateway	An external banking service integrated with the HMS via SSL/API to process patient payments. Supports full payment, partial payment, and account plan setup.
PaymentStatus	A field in the HMS Billing_Record table indicating the current state of a patient's account: Paid (settled in full), Owing (balance remaining), or Account (on a payment plan).
PDO (PHP Data Objects)	A PHP database access layer used throughout the HMS backend. PDO uses parameterised prepared statements to prevent SQL injection attacks and supports multiple database engines.
Pharmacist	A healthcare professional responsible for dispensing medication based on prescriptions issued by Doctors. In the HMS, Pharmacists receive digital prescription notifications and update inventory stock levels.
PHP	The server-side scripting language used to build the HMS application backend. PHP handles all business logic, FR module processing, session management, and database interaction.
POPIA (Protection of Personal Information Act)	South African data protection legislation (Act 4 of 2013) that governs how personal information must be collected, stored, processed, and protected. All HMS patient data handling complies with POPIA.
Prepared Statement	A pre-compiled SQL query template used in the HMS via PDO. User input is passed as a parameter separately from the SQL command, preventing SQL injection attacks.

Reorder Point	A predefined minimum stock level for an inventory item in the HMS. When StockLevel drops to or below the ReorderPoint, an early warning alert is automatically triggered (FR-05).
Role-Based Access Control (RBAC)	An HMS security model where system access is determined by the user's assigned role. Each role (SuperAdmin, Admin, Doctor, Nurse, Patient, Pharmacist) has a defined set of permissions enforced on every protected page.
SANC (South African Nursing Council)	The South African statutory body that regulates nursing professionals. The HMS validates Nurse license numbers against SANC standards during staff profile creation (FR-06).
SCRUM	An Agile framework used to manage the HMS development process. It organizes work into time-boxed sprints, with defined roles (developer, tester, admin) and regular feedback loops.
Sequence Diagram	A UML diagram showing the step-by-step interaction between system components for a specific use case, e.g. the sequence of events when a Doctor issues a prescription.
Service Worker	A JavaScript script that runs in the background of the HMS client browser, intercepting network requests and serving cached responses when the system is offline.
Session	A server-side mechanism used by the HMS to maintain a user's authenticated state across multiple page requests. Sessions are secured with HttpOnly and Secure cookie flags, and expire after 15 minutes of inactivity.
Sprint	A fixed time period (typically 1–2 weeks) in which a specific HMS module is developed and tested. The HMS is planned across 7 sprints from patient onboarding to offline-first backend logic.
SQL Injection	A type of attack where malicious SQL code is inserted into an input field to manipulate the database. The HMS prevents this entirely through PDO prepared statements used on all database queries.
SyncLog	A database table in the HMS that tracks all data entries made while the system is offline. Each record has a SyncStatus of either Local (cached) or Synced (uploaded to Azure).
Third Normal Form (3NF)	A database normalization standard applied to the HMS schema. It eliminates data redundancy by ensuring every non-key attribute depends only on the primary key and not on other non-key attributes.
TLS 1.3 (Transport Layer Security)	The cryptographic protocol used by the HMS to encrypt all data transmitted between the client browser, the web server, and the Azure cloud. TLS 1.3 is the current industry standard.
Transition Requirement (TR)	A requirement that addresses the migration of existing data into the new HMS system. TR-01 covers migrating fragmented physical and digital hospital records into the centralized HMS database.

UML (Unified Modelling Language)	A standardized visual modelling language used to design and document software systems. The HMS uses UML sequence diagrams (Section 3.5) and class diagrams (Section 4.3.4).
Work Breakdown Structure (WBS)	A hierarchical decomposition of the HMS project into manageable tasks across Planning, Design, Development, Testing, and Deployment phases.
XAMPP	A free, open-source local development environment used by the HMS development team. It bundles Apache (web server), MySQL (database), and PHP into a single installation for local testing.

Table 43: Glossary of Terms in HMS

8 Appendices B: Program Source Files

Below is the link to Richfield Account SharePoint
[HMS.zip](#)